



Министерство и науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Информационная система резервирования мест в ресторанах

Студент ИУ-5 85Б
(Группа)

(Подпись, дата) **И. И. Нагдимаев**
(И.О.Фамилия)

Руководитель ВКР

(Подпись, дата) **М.В. Чёрненький**
(И.О.Фамилия)

Нормоконтролер

(Подпись, дата) **Ю.Н. Кротов**
(И.О.Фамилия)

2022 г.

АННОТАЦИЯ

Расчётно-пояснительная записка квалификационной работы бакалавра содержит 51 страницу. С приложениями объем составляет 69 страниц. Работа включает в себя 31 иллюстраций, 7 таблиц и 2 фрагмента кода. В процессе выполнения было использовано 14 источников.

Объектом разработки является приложение для резервирования мест в заведениях общественного питания.

Данный продукт позволяет найти удобное заведение, свободное время и свободный столик.

Цель работы заключается в исследовании различных языков программирования, фреймворков и технологий.

При проведении бакалаврской квалификационной работы подробно изучалась специальность, учитывались методы, применимые к выбранной задаче, проводились эксперименты по реализации задачи по созданию информационной системы резервирования мест в заведениях общественного питания.

Пояснительная записка содержит 3 приложения.

СОДЕРЖАНИЕ

АННОТАЦИЯ.....	2
СОДЕРЖАНИЕ	3
ВВЕДЕНИЕ	5
1. КОНСТРУКТОРСКАЯ ЧАСТЬ	8
1.1. Формирование задач проектирования	8
1.2. Описание предметной области	9
1.2.1 Роли в системе.....	11
1.3 Архитектура автоматизированной системы.....	14
1.3.1 Архитектура клиентской части	16
1.3.2 Архитектура серверной части	20
1.4 Разработка модели базы данных.....	21
1.4.1 Инфологическая модель.....	21
1.4.2 Даталогическая модель базы данных	22
1.5 Выбранные технологии	23
1.5.1 Технологии клиентской части.....	23
1.5.2 Технологии серверной части	30
1.6 Основания выбора технологий	32
2. ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ.....	35
2.1 Тестирование клиентской части	35
2.1.1 Тестирование входа.....	35
2.1.2 Тестирование пользовательского функционала.....	39
2.1.3 Тестирование функционала администратора заведения	44

2.1.4 Тестирование функционала администратора	46
ЗАКЛЮЧЕНИЕ	49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	50
ПРИЛОЖЕНИЕ А ГРАФИЧЕСКИЕ МАТЕРИАЛЫ	52
ПРИЛОЖЕНИЕ В ТЕХНИЧЕСКОЕ ЗАДАНИЕ	54
ПРИЛОЖЕНИЕ С ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ	61

ВВЕДЕНИЕ

В США значительная часть посетителей ресторанов знакомы с сервисами онлайн-бронирования — по данным статистики, более 37% гостей хотя бы пару раз в своей жизни ими пользовались.

Рынок подобных приложений в Штатах также весьма развит, показателем чего, к примеру, являются сделки по покупкам подобных проектов более крупными ИТ-компаниями. К примеру, не так давно индийский сервис поиска ресторанов Zomato за \$52 млн выкупил NexTable, американскую онлайн-платформу бронирования столиков в ресторанах. Это было сделано для того, чтобы составить конкуренцию сервисам OpenTable от Priceline и SeatMe от Yelp.

В России и странах СНГ подобные инструменты пока не пользуются столь серьезной популярностью [1]. Но разобраться в том, зачем ресторанам вообще работать с сервисами бронирования, что это может им дать, и каковы риски, довольно интересно. На сегодняшнее время создаются тысячи сервисов для комфортной жизни людей, от самых ненужных мелочей в виде сервисов для наблюдения за работающими людьми или генерации лиц, до важных приложений, без которых представить человеческий мир довольно трудно, например, доставка еды. Человеческие потребности никогда не меняются. Одно из самых важных человеческих потребностей является еда. Самая очевидная мотивация владельцев ресторанов, толкающая их на сотрудничество с системами онлайн-бронирования — это надежда на то, что с их помощью заведению удастся привлечь новых посетителей. Количество подобных сервисов довольно велико даже на российском рынке — у нас есть gettable.ru, stoliki.ru, restotube.ru и т.п.

Подобная конкуренция заставляет такие проекты внимательнее относиться к требованиям ресторанов — например, добавлять возможности просмотра аналитической информации по бронированиям. При этом такие стартапы часто имеют собственный маркетинговый бюджет на привлечение

аудитории, что позволяет ресторанам рассчитывать на то, что о них узнают пользователи, которые раньше не слышали про данное конкретное заведение. Ресторанный бизнес один из самых прибыльных. Каждое заведение общественного питания делает свое приложение, в котором могут добавить просто меню либо же взаимодействовать непосредственно с рестораном, заказывать еду или же резервировать места. Для этого заведения общественного питания имеют свою карту мест и им несложно создать приложение для резервации даже при смене посадки. Но на просторах интернета нельзя увидеть одно общее приложение для агрегирования ресторанов, так как это практически невозможно, в отличие от агрегатов кинотеатров, где можно выбрать место или же агрегатов билетов на самолет, где в том же интерфейсе можно выбрать место в самолёте, так как места часто меняются и нужно работать с каждым рестораном. Сейчас появляются тысячи разных заведений общественного питания различных по уровню, типу, кухне, размерам, скорости обслуживания и многим другим аспектам. Потребителю довольно затруднительно сделать выбор. Поэтому задачей данной работы является конструирование и разработка информационной системы, позволяющей инструментами веб-разработки разработать программное обеспечение, удовлетворяющее потребностям пользователя в выборе заведения, резервировании столиков и выборе позиций в меню. Данное должно не только показывать ближайшие рестораны и их меню, но и резервировать места, которые задаются администратором. Заказывать заранее блюда, которые будут начинать готовиться до прихода клиента, принимать и отменять заказы в роли ресторана. И весь процесс должен проходить в рамках одного интерфейса и одного приложения.

Для достижения поставленной цели сформулированы следующие задачи:

- 1) Исследовать предметную область, определить функциональные;
- 2) Выбрать архитектуру программного обеспечения;
- 3) Разработать структуру программного обеспечения;
- 4) Собрать данные предметной области;

- 5) Реализовать веб-приложение;
- 6) Провести тестирование информационно - программного продукта;
- 7) Провести отладку программного продукта;
- 8) Оформить техническую документацию;

1. КОНСТРУКТОРСКАЯ ЧАСТЬ

1.1. Формирование задач проектирования

Назначение работы – разработка программного обеспечения, позволяющего пользователям резервировать места в заведениях общественного питания, а также предварительно выбирать позиции в меню.

Цель работы – конструирование и разработка информационной системы, позволяющей инструментами веб-разработки разработать программное обеспечение, удовлетворяющее потребностям пользователя в выборе заведения, резервировании столиков и выборе позиций в меню. Исследовать предметную область, определить функциональные задачи. Выбрать архитектуру программного обеспечения. Разработать структуру программного обеспечения. Собрать данные предметной области. Реализовать веб-приложение.

1.2. Описание предметной области

Анализ предметной области – главная задача любой работы. В первую очередь стоит отметить дизайн и отличия собственного веб-приложения. На основе других подобных сайтов можно определить вектор направления разработки, создания и улучшения приложения. Самая очевидная мотивация владельцев заведений общественного питания, толкающая их на сотрудничество с системами онлайн-бронирования — это надежда на то, что с их помощью заведению удастся привлечь новых посетителей. Количество подобных сервисов довольно велико даже на российском рынке — у нас есть gettable.ru, stoliki.ru, restotube.ru и т.п. Второй популярный способ использования сервисов бронирования — это привлечение аудитории для заполнения заведения в так называемые «тихие» часы, когда большая часть столиков пустует. С помощью небольших скидок или комплиментов от шеф-повара, которые предлагаются пользователям таких сервисов, заведение стимулирует их к осуществлению бронирования. В таком случае, бизнес практически ничего не теряет — если пользователь затем не придет, это не приведет к убыткам, поскольку посетителей в это время и так почти нет. А вот если он придет и что-то закажет, то заведение сразу получит дополнительный доход. В данной работе главным функционалом являются же именно бронирования столиков. Таких приложений довольно мало, так как это сделать практически невозможно с точки зрения бизнеса. Приходится договариваться с заведениями, делать для них формы и часто менять места. В большинстве сервисов предоставляют только время бронирования, а остальная договорённость происходит по телефону с хостесом.

Приведу пример довольно популярного сервиса в США Tock. Вкратце, работает это так — все похоже на покупку билета на концерт: пользователь выбирает дату и время, ресторан, а затем оплачивает все, что будет водить в заказ. В итоге затем клиент просто приходит в заведение и получает то, за что уже заплатил.

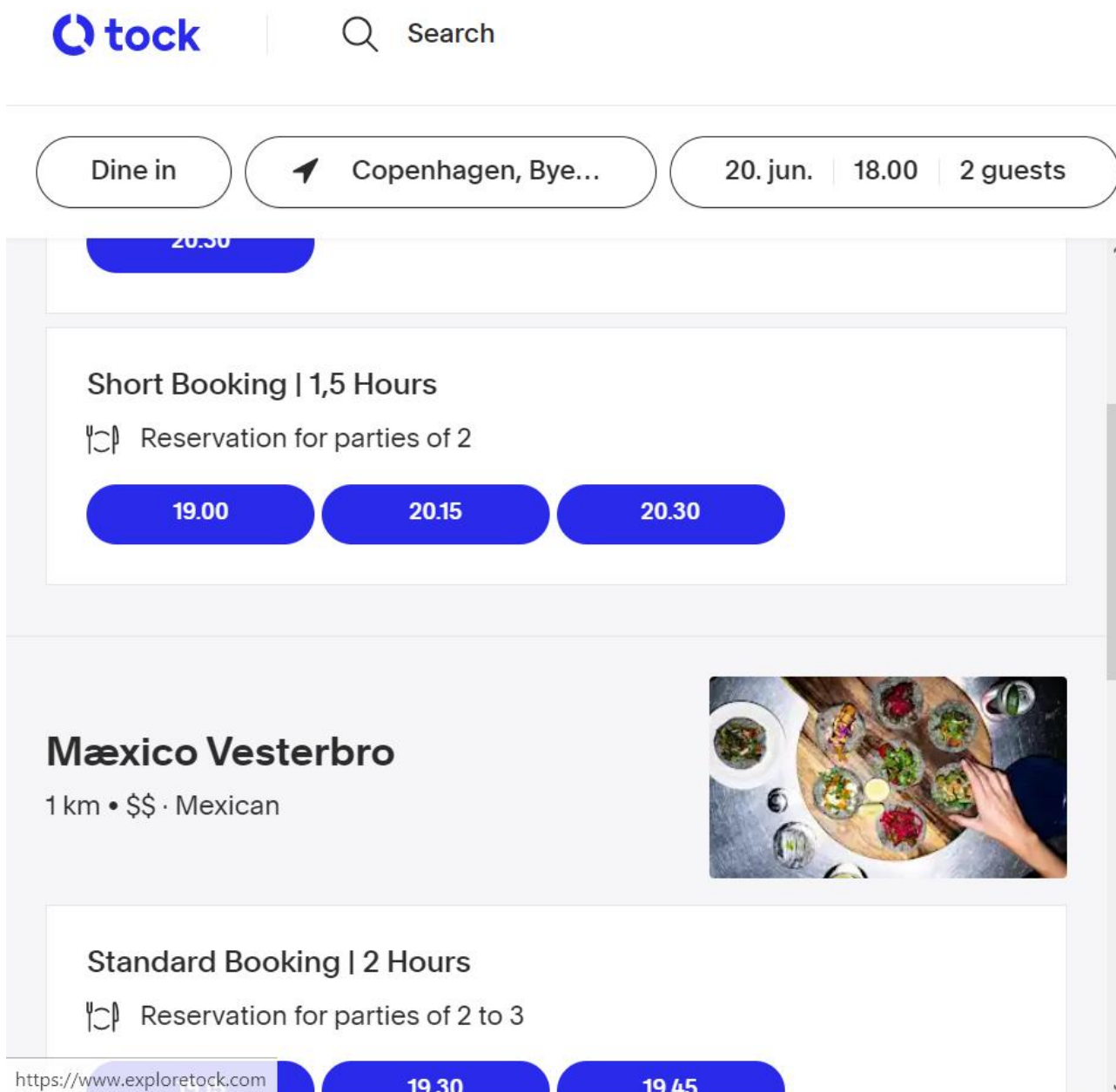


Рис. 1. Сервис для бронирования Tock

Как уже было выше сказано, в данном сервисе нужно выбрать блюда заранее и оплатить.

В моем приложении я позволяю пользователю как заказывать, будучи в ресторане, так и заранее. Анализируя данную область стало понятно, что без какого-либо своего инструмента по построению карт заведения общественного питания не обойтись, иначе это было бы очень долго и при какой-либо перестановке администратор может поменять в считанные секунды. Также администратор с помощью своей администраторской панели добавлять сами рестораны и блюда. Определённо существовала проблема во

продолжительности бронирования. Человек может уйти и через полчаса и засидеться на пол дня, если же это банкет. Для этого я реализовал роль администратора ресторана или же персонала. В роли персонала можно отменять бронирования, изменять заказ (т.е. увеличивать или уменьшать позицию, а также добавлять или удалять новые позиции меню в заказ пользователя). Также требуется и сама авторизация, добавление роли администратора или же роли персонала добавляются самим разработчиком на удалённом сервере. Аутентификация происходит посредством наименования электронной почты и пароля, который должен подходить требованиям сервиса. Так как в ходе брони могут добавляться заказы, оплату я сделал на месте после трапезы, как это и происходит в обычных ресторанах.

В заключении необходимо обобщить анализ предметной области и самого веб приложения.

Основным отличием от других веб-приложений будет понятность и гибкость для пользователя. Приложение не будет отправлять и пересылать на отдельные сервисы самого ресторана. Весь функционал от выбора ресторана до выхода из него будет происходить посредством одного интерфейса и одного сервиса.

Соответственно приложение, должно быть удобным и понятным для выбора ресторана, резервирования столиков и выбора блюд.

1.2.1 Роли в системе

Анализ предметной области выявил несколько различных по задачам типы пользователей, всех действующих в этом процессе можно разделить на:

1. Пользователя
2. Сотрудника заведения
3. Администратора

Каждая из ролей имеет доступ только к определенному функционалу в системе для максимально эффективного выполнения своих обязательств. Обычно в разделении по ролям функции администратора включают функции

ролей ниже и по аналогии с нижними ролями. Но в моем приложении практически нет схожего функционала у каких-либо ролей, все разные.

Также представлена UML диаграмма прецедентов на рисунке 1 и на рисунке 2

Прецедент	Действующее лицо
Аутентификация	Пользователь, Сотрудник ресторана, Администратор
Выход	Пользователь, Сотрудник ресторана, Администратор
Регистрация	Пользователь
Показ доступных ресторанов	Пользователь, Администратор
Поиск ресторана по названию	Пользователь, Администратор
Выбор времени бронирования	Пользователь
Бронирование места	Пользователь
Добавление, удаление изменение блюд в предоставленном заказе	Пользователь, Сотрудник ресторана
Отмена заказа	Пользователь, Сотрудник ресторана
Сохранение понравившихся ресторанов	Пользователь, Сотрудник ресторана
Просмотр заказа	Пользователь, Сотрудник ресторана
Добавление нового ресторана	Администратор
Редактирование информации о ресторане	Администратор
Добавление, удаление изменение блюд в ресторане	Администратор
Добавление, удаление, изменение мест в ресторане	Администратор
Просмотр ближайших заказов ресторана	Сотрудник ресторана

Рис. 2. Таблица диаграммы прецедентов UML

Пользователь

Пользователем является непосредственно клиент. Он может регистрироваться, выбирать любимые заведения, искать по названию, бронировать время и место, выбирать блюда.

Изначально регистрируясь в системе каждому клиенту присваивается роль пользователя.

Пользователь может совершать следующие действия:

1. Регистрация
2. Поиск заведения по названию
3. Выбор времени
4. Выбор места
5. Создание заказа
6. Просмотр заказов
7. Отмена брони
8. Добавление, удаление, изменение заказа
9. Изменение профиля
10. Выход из системы

Сотрудник заведения

Сотрудник заведения является отдельной ролью. Она задается разработчиком посредством работы с удаленной базой данных. Сотрудник заведения владеет всем управленческим функционалом в рамках одного заведения общественного питания.

Уполномоченное лицо может совершать следующие действия:

1. Просмотр ближайших заказов клиентов
2. Аутентификация
3. Добавление, удаление, изменение заказов.
4. Отмена брони.
5. Выход из системы

Администратор

Роль администратора является самой продвинутой и обладает всем управленческим функционалом. Роль также задается разработчиком через непосредственную работу с удаленной базой данных

Уполномоченное лицо может совершать следующие действия:

- 1) Аутентификация.
- 2) Добавление, удаление заведений.
- 3) Изменение информации о заведении.
- 4) Добавление, удаление позиций в меню заведения.
- 5) Изменение информации о позиции в меню заведения.
- 6) Добавление, удаление изменение мест заведения.
- 7) Выход из системы

1.3 Архитектура автоматизированной системы

Для разработки программного продукта необходимо использовать трехуровневую чистую архитектуру. Эта архитектура обеспечивает разделение ответственности между компонентами системы, позволяя разделить ответственность между ними и масштабировать систему на любой из уровней по мере необходимости.

Клиентский слой представлен как часть интерфейсной части комплекса, доступной любому пользователю. Если у вас есть доступ к базам данных, вам не следует их скачивать. Это зависит от основных подходов бизнес-логики (из соображений безопасности и масштабируемости), оно должно сохранять состояние приложения на протяжении всей жизни. Он представляет собой простой уровень бизнес-логики: интерфейс авторизации, алгоритмы шифрования и проверки вводимых значений (соответствие формата), простые операции с уже загруженными на терминал данными.

Второй уровень включает локальное хранилище приложений. Именно здесь сосредоточена большая часть бизнес-логики. Снаружи только фрагменты,

экспортируемые в клиент и логические элементы для интеграции с БД. Для реализации этого компонента используется промежуточное ПО. Для создания серверов приложений, спроектированных таким образом, я могу добавлять дополнительные экземпляры и повышать производительность программного комплекса без изменения кода приложения.

Сервер базы данных обеспечивает хранение данных и размещается на отдельном уровне, обычно реализуемом СУБД, подключение к этому компоненту обеспечивается только уровнем сервера приложений, например, через драйвер Go и библиотек.

В самых простых конфигурациях все компоненты или некоторые из них могут быть объединены на одном вычислительном узле. В продуктивных конфигурациях для сервера базы данных или кластера серверов баз данных обычно используется выделенный вычислительный узел, а для серверов приложений — выделенная группа вычислительных узлов, к которым клиенты подключаются напрямую. Полученная трехуровневая архитектура представлена на рисунке 3.

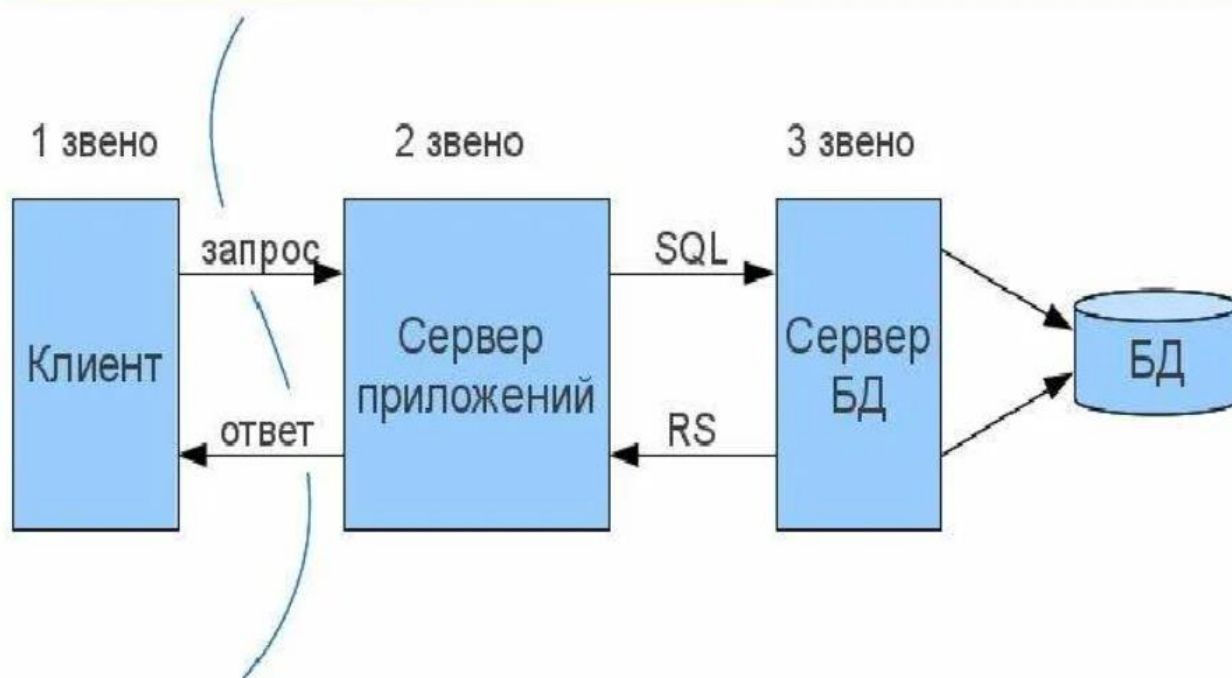


Рис. 3. Архитектура системы

1.3.1 Архитектура клиентской части

С точки зрения архитектуры сайта веб-приложение состоит из двух частей: клиентской и серверной. Как и в случае с интерфейсом, клиентская часть называется интерфейсом. С точки зрения пользователя это то, что он видит на экране устройства. Серверная часть или бэкенд — это программно-аппаратный комплекс, обеспечивающий работу сервиса. Они представляют собой набор инструментов, реализующих логику приложения.

Выделяют три вида веб-приложений, которые определяют подход к разработке:

SPA или одностраничное приложение [2] — это одностраничный веб-сайт, который загружается на одной HTML-странице. Благодаря динамическим обновлениям с помощью JavaScript пользователям не нужно перезагружать или загружать дополнительные страницы. Пользователь видит весь контент в браузере, и когда он прокручивает страницу или переходит на другие сайты, нужные элементы просто добавляются.

Фронтэнд, т.е. клиент представляет собой интерактивную часть программы, которая выполняется в веб-браузере на компьютере, смартфоне или планшете. Клиентская часть реализует пользовательский интерфейс веб-приложения и загружается в виде динамических веб-страниц.

Рассмотрим основные компоненты стека интерфейсов.

HTML — это стандартный язык разметки, используемый для создания веб-проектов. Эти компоненты позволяют отображать стандартные страницы и отображать форматированный текст, изображения, таблицы или формы ввода данных. Когда браузер отправляет запрос на URL-адрес для получения страницы или приложения с целью получения страницы или приложения, сначала возвращается HTML-документ. HTML-документ может предоставить вам дополнительную информацию о дизайне или макете в виде CSS, а также об использовании JavaScript.

По умолчанию CSS — это язык разметки для определения стиля и макета HTML-элементов. HTML и CSS обеспечивают структуру, а HTML и CSS обеспечивают стиль. CSS определяет шрифты, цвета и стили для отдельных элементов, а также положение отдельных элементов на разных устройствах.

JavaScript — это язык программирования, который помогает реализовать сложное поведение веб-страницы. В большинстве случаев JavaScript используется для создания привлекательных интерактивных элементов для веб-страниц, которые улучшают взаимодействие с пользователем. С помощью JavaScript вы можете создавать меню, анимацию, видеоплееры, интерактивные карты и даже простые браузерные игры.

Для более эффективной разработки JavaScript разработчики используют фреймворки, которые также являются важной частью стека технологий. Фреймворк — это рабочая среда, тип каркаса, помогающий более эффективно создавать и поддерживать программные продукты. Далее мы перечислим некоторые фреймворки и библиотеки, которые мы обычно используем в проектах.

MSPA[3] или многостраничное приложение — это многостраничные приложения, в которых каждый раз при изменении данных или загрузке новой информации страница обновляется.

PWA или Progressive Web App — это своего рода гибрид веб-сайта с мобильным приложением. Их можно установить на главный экран смартфона, отправлять push-уведомления и работать в автономном режиме.

Так как это приложение требует отзывчивости и мобильности, я выбрал SPA. Исследования показывают, что пользователи чувствительны к скорости загрузки страниц. По скорости загрузки SPA-сайты не имеют себе равных, что является их главным преимуществом. Поскольку львиная доля ресурсов загружается за один сеанс, при дальнейших действиях SPA быстро обновляет необходимую часть данных, что существенно экономит время.

Однако высокая скорость — не единственный плюс SPA приложения. Помимо этого, одностраничные приложения имеют следующие преимущества:

легкость создания. Библиотеки и фреймворки уже доступны для разработки SPA-приложения, а работу над фронтендом и бэкендом можно вести параллельно. Кроме того, на основе готового кода в дальнейшем можно создать мобильное приложение.

Гибкость пользовательского интерфейса. Для одной страницы проще разработать привлекательный и интерактивный дизайн интерфейса;

Пользователь будет переходить по вкладкам (тапбару) и не будет ждать от сервера новую HTML страницу, а сразу переходить посредством языка Javascript, что намного ускорит наше приложение.

При создании пользовательского интерфейса используется шаблон Flux - шаблон программирования, который сочетается с реактивным программированием. Flux [4] строится на однонаправленном потоке данных. Схема шаблона FLUX представлена на рисунке 4.

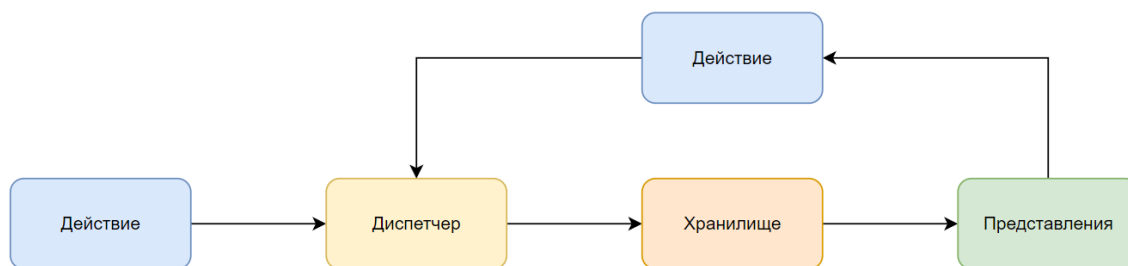


Рис.4. Схема шаблона Flux

Ключевой особенностью Flux является односторонняя передача данных между компонентами архитектуры Flux. В частности, настройка архитектуры предусматривает ограничение потока данных за счет исключения возможности обновления состояния самих компонентов. Это делает поток данных предсказуемым и облегчает понимание причин возможных программных ошибок.

Flux в основном имеет дело с информационной архитектурой, которая затем отражается в архитектуре программного обеспечения, поэтому уровень представления слабо связан с другими уровнями системы.

Действия

Действия являются выражением событий. Затем диспетчеры передают информацию об этих действиях подчиненным субъектам (бизнесу). Это означает, что новый шаг выполняется только после того, как предыдущий был полностью обработан компонентами. Динамический относится к синхронизированному процессу. Также имя может быть полезным дополнением к его информации.

Диспетчер

Диспетчер служит для передачи информации о действиях в память. Однако в упрощенном варианте диспетчер может быть единственным для всего приложения. Обычно диспетчер памяти регистрирует свои функции обратного вызова и зависимости от памяти.

Хранилища

Содержимое памяти находится там, где находится состояние приложения. По мнению Flux, остальные компоненты «устарели», распознавание и изменение статуса магазина происходит строго на основе данных кампании и старого статуса из магазина.

Представления

Представление — это элемент, отвечающий за предоставление информации пользователю. Архитектура Flux технически вообще не имеет отношения к внутренней части представлений. Это конечная точка потока данных в этой информационной области. Только для информационной архитектуры важно только то, что данные попадают в систему только через действия.

1.3.2 Архитектура серверной части

В серверной части реализуется основная логика приложения. Здесь происходит вся бизнес-логика. Добавление, удаление и изменение в удалённой базе данных посредством языка программирования и СУБД.

В качестве архитектуры я выбрал монолитное приложение вместо микросервисной архитектуры. Большим преимуществом монолита является то, что его легче реализовать. В монолитной архитектуре вы можете быстро начать реализовывать свою бизнес-логику, вместо того чтобы тратить время на размышления о межпроцессном взаимодействии.

Говоря об операциях, важно сказать, что монолит прост в развертывании и легко масштабируется. Для развертывания вы можете использовать скрипт, загружающий ваш модуль и запускающий приложение. Масштабирование достигается путем размещения Loadbalancer перед несколькими экземплярами вашего приложения.

В качестве архитектуры программного обеспечения была выбрана чистая архитектура [5] с разделением программы на часть delivery, usecase, domain и repository. Схема данной архитектуры представлена на рисунке 5.

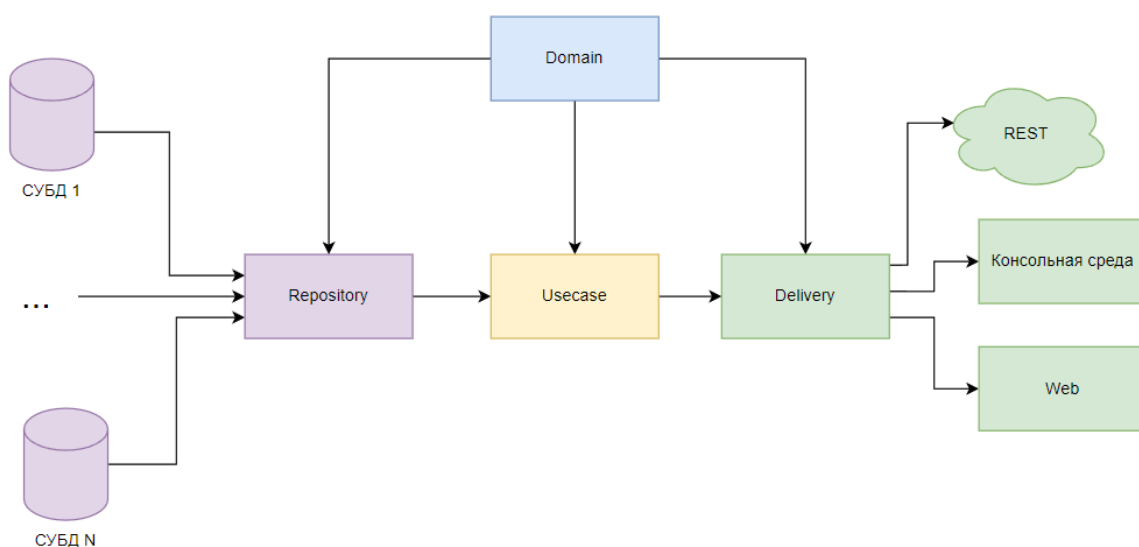


Рис. 5. Схема чистой архитектуры

1.4 Разработка модели базы данных

Приложение должно хранить большое количество данных. База данных позволит сохранять все предпочтения пользователей. Координаты столиков, огромное количество заказов. Так как множество данных приходилось высчитывать из данных БД, мне пришлось денормализовать некоторые таблицы для быстрого получения данных. Например, нужно было смотреть все столики заведения и найти самый максимальный этаж у заведения, поэтому пришлось добавить поле количества этажей самого заведения общественного питания.

1.4.1 Инфологическая модель

Каждая сущность в системе также имеет поле ID для связи с другими сущностями.

1. Пользователь

Атрибуты:

- 1) Электронная почта
- 2) Пароль
- 3) Полное имя
- 4) Номер телефона
- 5) Флаг админа
- 6) Флаг сотрудника заведения

2. Заведение

Атрибуты:

- 1) Название
- 2) Описание
- 3) Адрес
- 4) Название ближайшего метро
- 5) Время открытия
- 6) Время закрытия
- 7) Тип кухни
- 8) URL на иконку заведения
- 9) Количество этажей

3. Место

Атрибуты:

- 1) Вместимость
- 2) Номер
- 3) Левый отступ от левого края карты
- 4) Верхний отступ от верхнего края карты

- 5) Ширина стола
 - 6) Длина стола
 - 7) Номер этажа
 - Связи:
 - 8) Заведение M:1
4. Заказ
- Атрибуты:
- 1) Время начала бронирования
 - 2) Время конца бронирования
 - 3) Стоимость заказа
 - 4) Время создания
 - 5) Статус заказа
- Связи:
- 6) Пользователь M:1
 - 7) Заведение M:1
 - 8) Блюдо M:M
5. Оплата
- Атрибуты:
- 1) Общая стоимость
 - 2) Формат оплаты
 - 3) Статус
- Связи:
- 4) Заказ 1:1
6. Блюдо
- Атрибуты:
- 1) Название
 - 2) Описание
 - 3) Цена
- Связи:
- 4) Заказ M:M
 - 5) Заведение M:1

Данная форма представлена в приложении А1

1.4.2 Даталогическая модель базы данных

По этой модели мы реализуем вышеописанную модель в СУБД PostgreSQL. Каждая сущность, представленная в информационной модели, представляет собой таблицу PostgreSQL на диаграмме. Диаграмма даталогической модели базы данных представлена в приложении А2.

Сама СУБД у меня разворачивается Docker контейнере независимо от бэкенда. Поэтому БД можно развернуть на отдельном сервере или вынести его в отдельный репозиторий со своим Go скриптом для миграций и со своим дампом.

1.5 Выбранные технологии

Для реализации программной части каждого из элементов автоматизированной системы выбраны следующие технологии. Все приложения состоят из использования технологических решений и технологий, используемых для удовлетворения этих архитектурных или функциональных потребностей. Клиенты и серверы имеют совершенно разные программы, которые общаются друг с другом через HTTP-запросы.

Клиентская часть:

- 1) Язык программирования Javascript
- 2) Фреймворк React
- 3) Метаязык SCSS
- 4) Веб-сервер NGINX
- 5) Canvas

Серверная часть:

- 1) Язык программирования Golang
- 2) СУБД PostgreSQL
- 3) СУБД Tarantool
- 4) Контейнеризатор приложений Docker

1.5.1 Технологии клиентской части

Язык программирования JavaScript

JavaScript [6] — мультипарадигменный язык программирования. Поддерживает объектно-ориентированный, императивный и функциональный стили. Является реализацией спецификации ECMAScript.

JavaScript широко используется как встраиваемый язык для программного доступа к объектам приложения. Чаще всего он используется в браузерах в качестве языка сценариев для создания интерактивных веб-страниц.

Ключевые особенности архитектуры: динамическая типизация, слабая типизация, автоматическое управление памятью, прототипное программирование, функции как первоклассные объекты.

JavaScript — это объектно-ориентированный язык, но прототипирование, используемое в языке, отличает работу с объектами от традиционных языков, ориентированных на классы. Кроме того, JavaScript обладает рядом свойств, присущих функциональным языкам — функции как первоклассные объекты, объекты как списки, каррирование, анонимные функции, замыкания — придающие языку дополнительную гибкость.

Фреймворк React

React[7] (иногда React.js или ReactJS) — JavaScript-библиотека с открытым исходным кодом для разработки пользовательских интерфейсов.

React разрабатывается и поддерживается Facebook, Instagram и сообществом отдельных разработчиков и корпораций.

React[8] можно использовать для разработки одностраничных и мобильных приложений. Его цель — обеспечить высокую скорость, простоту и масштабируемость. В качестве библиотеки разработки пользовательского интерфейса React обычно используется с другими библиотеками, такими как MobX, Redux и GraphQL. Еще одна важная часть Vue.js — реактивность. Представления меняются по мере изменения данных, что позволяет динамически обновлять часть страницы без полной перезагрузки. Поскольку области в системе являются динамическими, страницы часто обновляются, и очень полезно изменять только небольшую часть страницы. Vue также реализует маршрутизатор Vue, который обеспечивает формальную навигацию по

страницам приложения на клиенте, оставаясь при этом одностраничным приложением.

Angular имеет двунаправленную систему привязки данных. Это проявляется, например, в том, что изменение формы элемента приводит к автоматическому обновлению статуса заявки. Это затрудняет отладку и является основным недостатком этой платформы. Если при таком подходе что-то пойдет не так, программист не сможет точно знать, что вызвало изменение состояния приложения.

В React, с другой стороны, используется односторонняя привязка данных. Это — большой плюс данной библиотеки, так как выражается это в том, что программист всегда точно знает о том, что привело к изменению состояния приложения. Подобный подход к привязке данных значительно упрощает отладку приложений.

В React все компоненты пользовательского интерфейса могут быть выражены в виде наборов чистых функций. Использование чистых функций для формирования UI можно сравнить с глотком чистого воздуха.

Метаязык SCSS

SCSS [9] — модуль, включенный в Haml. SCSS — это метаязык для повышения уровня абстракции кода CSS и упрощения каскадирования файлов таблиц стилей.

SCSS — использует фигурные скобки, как и сам CSS, SCSS — это «диалект» языка SCSS. SCSS — это язык, похожий на HAML, но предназначенный для упрощения создания кода CSS. Проще говоря, SCSS — это язык, код которого переводится в обычный код CSS специальной программой Ruby. Синтаксис этого языка очень гибкий, он учитывает множество мелочей, которые так желательны в CSS. Я использую SCSS, так как он позволяет перенести весь CSS код для всех браузеров. Также SCSS позволяет правильно задавать медиа

запросы, чтобы браузер правильно понимал мой код. Также он достаточно Хор. оптимизирует код CSS.

Фрагмент кода.

```
html{
    background: #6247AA;
}

body {
    width: 100vw;
    height: 100vh;
    margin: 0px;
}

.app {
    display: flex;
    flex-direction: row;
    width: 100%;
    height: 100%;
    overflow: hidden;

    &__tapbar{
        display: flex;
        flex-direction: column;
        min-width: 90px;
        max-width: 110px;
    }
    &__body{
        display: flex;
        flex-direction: column;
        justify-content: center;
        width: 100%;
        height: 100%;

        &_center {
            align-items: center;
        }
    }
}
```

Веб-сервер NGINX

Nginx — веб-сервер и почтовый прокси-сервер, работающий на Unix-подобных операционных системах.

Связь в nginx охватывает несколько различных соединений и осуществляется через вызовы операционной системы select, epoll (Linux) или kqueue. При обработке событий от дескрипторов рабочий процесс обрабатывает события от дескриптора. Информация, полученная от клиента, обрабатывается в конечном автомате. Для каждого последующего запроса указанная конфигурация обрабатывается цепочкой модулей. Клиентский вызов формируется на основе ответа клиента в буфере данных, который хранит информацию о файлах или памяти, либо указывает на часть файла. Буферы связаны цепочками, определяющими порядок передачи данных клиенту. Если вам нужны эффективные операции ввода-вывода, такие как writev и sendfile, nginx использует их везде.

Так как у нас и клиентская, и серверная часть находятся на одном сервере. Нам потребовался веб сервер, который мог быстро и понятно пересылать через 80 порт, который находится по умолчанию. Иначе CORS Policy не давало приложению посылать запросы на 8000, который слушает серверная часть. Также портом по умолчанию для React является 3000. Поэтому Nginx позволит пользователю не вбивать дополнительно через разделитель “:” порт 3000. А просто вбить адрес сервера. Так как по умолчанию запрос идет на 80 порт Nginx пересылает его на 3000. А от клиентской части запросы идут на 80 который перенаправляет их на 8000 порт бэкэнда. Также Nginx позволит балансировать между несколькими серверами при масштабировании.

Фрагмент кода.

```
include /etc/nginx/main.d/*.conf;  
# user nginx;  
# worker_processes auto;
```

```

error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;
events {
    worker_connections 1024;
    # multi_accept on;
}
http {
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    ssl_protocols TLSv1 TLSv1.1 TLSv1.2 TLSv1.3;
# DroppingSSLv3, ref: POODLE
    ssl_prefer_server_ciphers on;
    access_log /var/log/nginx/access.log;
    error_log /var/log/nginx/error.log;
    include /etc/nginx/conf.d/*.conf;
    include /usr/share/nginx/modules/*.conf;
    upstream front {
        server localhost:3000;
    }
    upstream back {
        server 127.0.0.1:8000;
    }
    server {
        listen 80;

```

```

    access_log /var/log/nginx/Drip.access.log;
    error_log /var/log/nginx/Drip.error.log;
    client_max_body_size 20M;
    location /api/v1/ {
        proxy_pass http://back;
        proxy_set_header Host $host;
        proxy_set_header
X-Forwarded-For$proxy_add_x_forwarded_for;
    }
    location / {
        proxy_pass http://front;
        proxy_set_header Host $host;
        proxy_set_header
X-Forwarded-For$proxy_add_x_forwarded_for;
    }
    sendfile on;
}
}

```

Canvas

Canvas [14] — элемент HTML5, предназначенный для создания растрового двухмерного изображения при помощи скриптов, обычно на языке JavaScript. Отсчет начинается с левого верхнего угла, где он расположен. С его помощью строятся все элементы блоков, входящие в состав блока. Размеры координатного пространства не обязательно отражают размер реальной области. На самом деле ширина 300 пикселей, а высота 150 пикселей.

Этот инструмент используется для создания графики в статьях и игрового поля некоторых браузерных игр. С его помощью можно встроить видео на страницу и создать полноценный плеер.

WebGL используется для 3D-графики посредством аппаратного ускорения.

11 февраля 2013 года компания Google выпустила JavaScript-библиотеку `explorercanvas`, позволяющую работать с Canvas в браузерах IE7 и IE8.

Canvas я использовал для построения карты. Существовало 2 варианта построения карты:

1. Двигать сами элементы HTML с помощью CSS и задавать им отступы динамически.
2. Вставить тэг `canvas` и отрисовать пикселями.

Так как мне нужно было не просто отрисовывать, но и передвигать, я выбрал второй вариант, потому что реализовать такой функционал на тэгах крайне сложно и не целесообразно. Поэтому я решил вставить отдельный HTML элемент `canvas` для отрисовки по пикселям. С помощью JavaScript `canvas` и React общаются между собой и управляют друг другом.

1.5.2 Технологии серверной части

Язык программирования Golang

Go[10] — компилируемый многопоточный язык программирования, разработанный внутри компании Google. С сегодняшнего дня поддерживается официальный компилятор языка, разработанный создателями.

СУБД PostgreSQL

PostgreSQL[11] — свободная объектно-реляционная система управления базами данных (СУБД).

СУБД Tarantool

Tarantool[12] — это платформа `in-memory` вычислений с гибкой схемой данных для эффективного создания высоконагруженных приложений. Включает в себя базу данных и сервер приложений на Lua.

Пользователь может динамически добавлять, удалять и изменять функции. Совместная многозадачность используется для одновременного запуска кода.. Для расширения сервера приложений доступны Lua библиотеки как из репозитория tarantool/rocks так и собранные из исходников.

Контейнеризатор приложений Docker

Docker[13] — это программа для автоматизации развертывания и управления приложениями в контейнерных средах, а также конструктор приложений. Это позволяет «упаковывать» приложения с различными средами и зависимостями в один контейнер, который можно развернуть во всех системах Linux, управляемых cgroups в ядре, и предоставляет набор команд для управления этими пакетами. Вначале он использовал возможности LXC и только в 2016 году начал использовать свою библиотеку для виртуализации ядра Linux — libcontainer.

1.6 Основания выбора технологий

Для обоснования выбора будем использовать метод взвешенной суммы. Для перевода качественных параметров в количественные будем использовать Таблицу 1.

Таблица 1 – Перевод качественных параметров в количественные

Качество	Отл.	Хор.	Удов.	Пл.
Нормированный показатель	1	0.75	0.5	0.25

Выбор фреймворка

В Таблице 2 представлены данные для сравнения трех наиболее популярных фреймворков для реализации клиентской части веб-приложений.

Таблица 2 – Исходные данные для выбора фреймворка клиентской части

	Весовой коэф.	React	Vue	Angular
Скорость разработки	0.3	Хор.	Отл.	Удов.
Понятность	0.1	Отл.	Отл.	Хор.
Мощность	0.5	Отл.	Хор.	Хор.
Наличие встроенных вспомогательных инструментов	0.4	Отл.	Хор.	Удов.

В Таблице 3 показаны нормированные значения.

Таблица 3

	Весовой коэф.	React	Vue	Angular
Скорость разработки	0.3	0.75	1	0.5
Понятность	0.1	1	1	0.75
Мощность	0.5	1	0.75	0.75
Наличие встроенных вспомогательных инструментов	0.4	1	0.75	0.5
Общий показатель		0.925	0.833	0.72

Таким образом для клиентской части был выбран фреймворк – React.

Системы сборки для клиентской части

В Таблице 4 показаны данные для противопоставления наиболее популярных сервисов сборки для клиентской части.

Таблица 4 – Исходные данные для выбора языка программирования API

	Весовой коэф.	Grunt	Gulp	Gear	Brunch
Понятность	0.2	Отл.	Отл.	Пл.	Хор.
Скорость	0.5	Пл.	Хор.	Отл.	Удов.
Методичка	0.4	Пл.	Хор.	Удов.	Хор.
Библиотеки	0.1	Отл.	Удов.	Пл.	Удов.

В таблице 5 указаны нормированные количественные значения.

Таблица 5

	Весовой коэф.	Grunt	Gulp	Gear	Brunch
Понятность	0.2	1	1	0.25	0.75
Скорость	0.5	0.25	0.75	1	0.5
Методичка	0.4	1	0.75	0.75	0.75
Библиотеки	0.1	1	0.25	0.25	0.5
Общий показатель		1.05	1.025	0.825	0.85

Таким образом в качестве системы сборки для клиентской части был выбран Gulp.

Выбор препроцессора CSS

В Таблице 6 представлены данные для сравнения двух наиболее популярных препроцессоров CSS.

Таблица 6 – Исходные данные для выбора препроцессора CSS

	Весовой коэф.	SASS	LESS
Понятность	0.2	Удов.	Отл.
Универсальность	0.4	Отл.	Хор.
Плагины	0.5	Отл.	Удов.
Мощность	0.4	Хор.	Удов.

В таблице 7 указаны нормированные количественные значения.

Таблица 7

	Весовой коэф.	SASS	LESS
Понятность	0.2	0.5	1
Универсальность	0.4	1	0.75
Плагины	0.5	1	0.5
Мощность	0.4	0.75	0.5
Общий показатель		1.275	0.825

Таким образом в качестве реляционной базы данных был выбран препроцессор для CSS SASS.

2. ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

2.1 Тестирование клиентской части

2.1.1 Тестирование входа

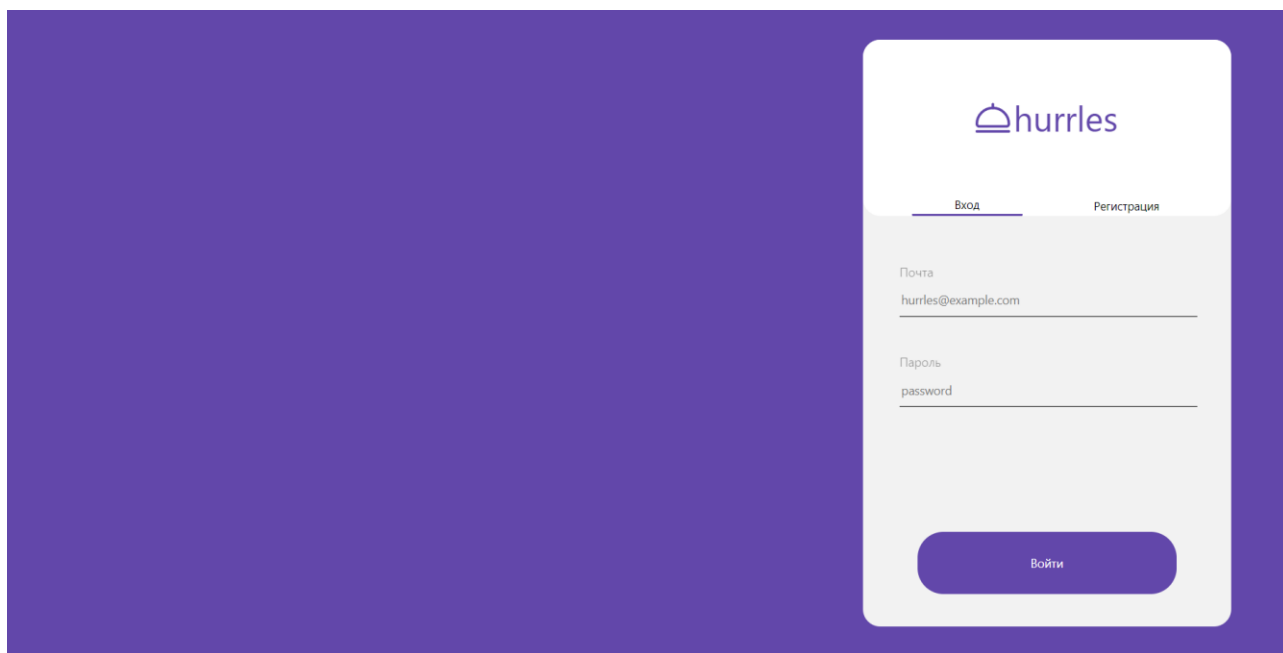


Рис. 6. Страница «Вход»

На этой странице отображаются две вкладки «Вход» и «Регистрация». По нажатию на «Регистрацию» форма входа переходит на страницу «Регистрация».

На вкладке «Вход» отображаются 2 поля: поле «Почта» и поле «Пароль». Здесь происходит аутентификация и авторизация, и приложение понимает роль пользователя.

Также существует валидация почты и пароля.

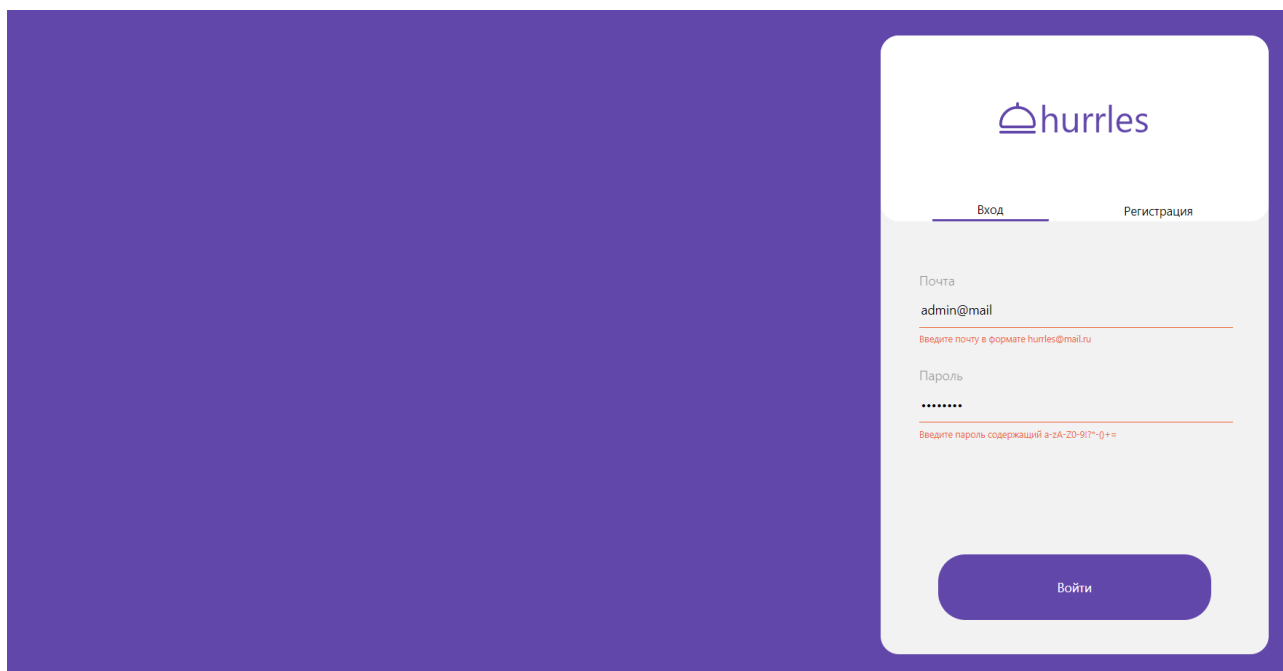


Рис. 7. Страница “Вход” с невалидными почтой и паролем

Если почты не существует в базе данных или же при правильной почте пользователь ввёл неподходящий пароль, то высвечивается ошибка.

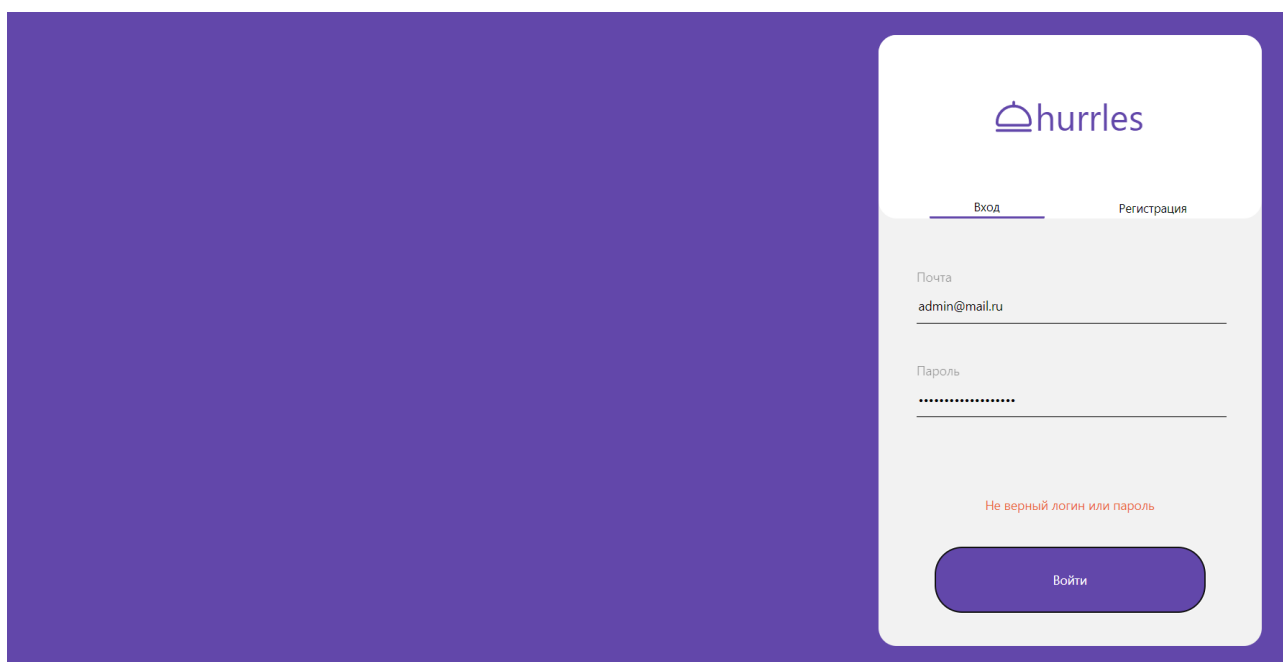


Рис. 8. Страница “Вход” с неправильным логином или паролем

При нажатии на кнопку «Регистрация» страница переходит на форму регистрации.

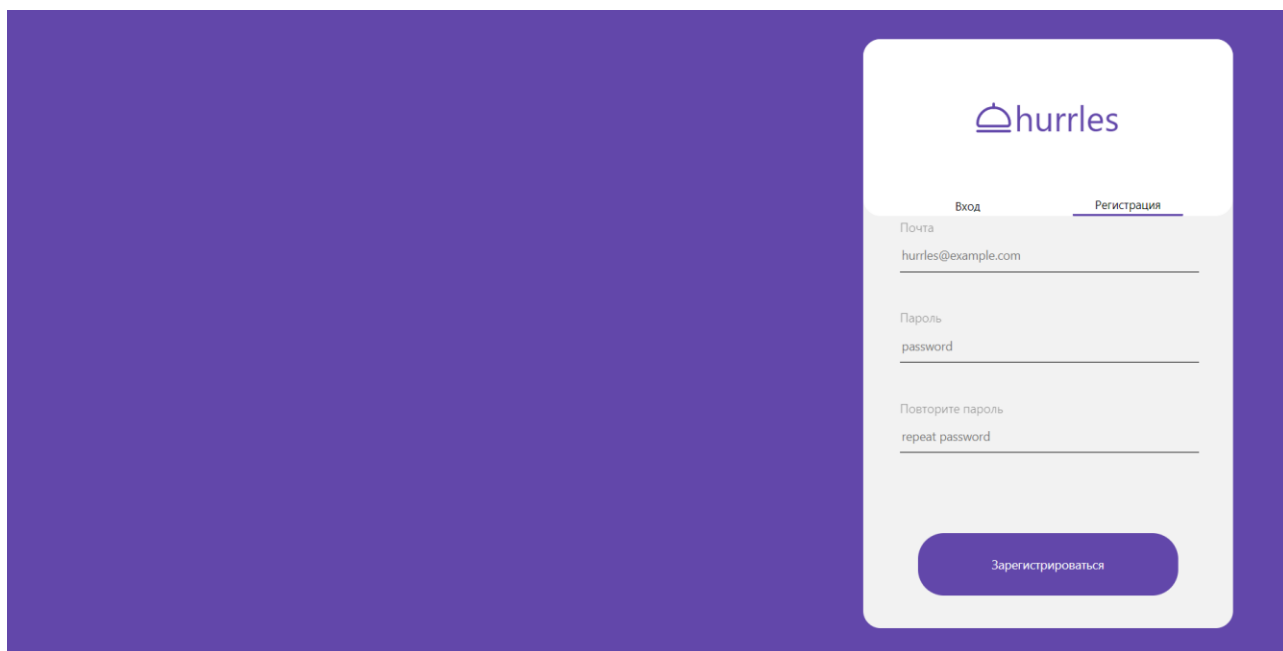


Рис. 9. Страница «Регистрация»

На странице «Регистрация» отображаются три поля: пол «Почта», поле «Пароль» и поле «Повторите пароль». Здесь пользователь может зарегистрировать себя и внести себя в базу данных на удалённом сервере. Также он задаёт пароль. Если в поле «Повторите пароль» данные не совпадают с данным в поле «Пароль», то выводится ошибка.

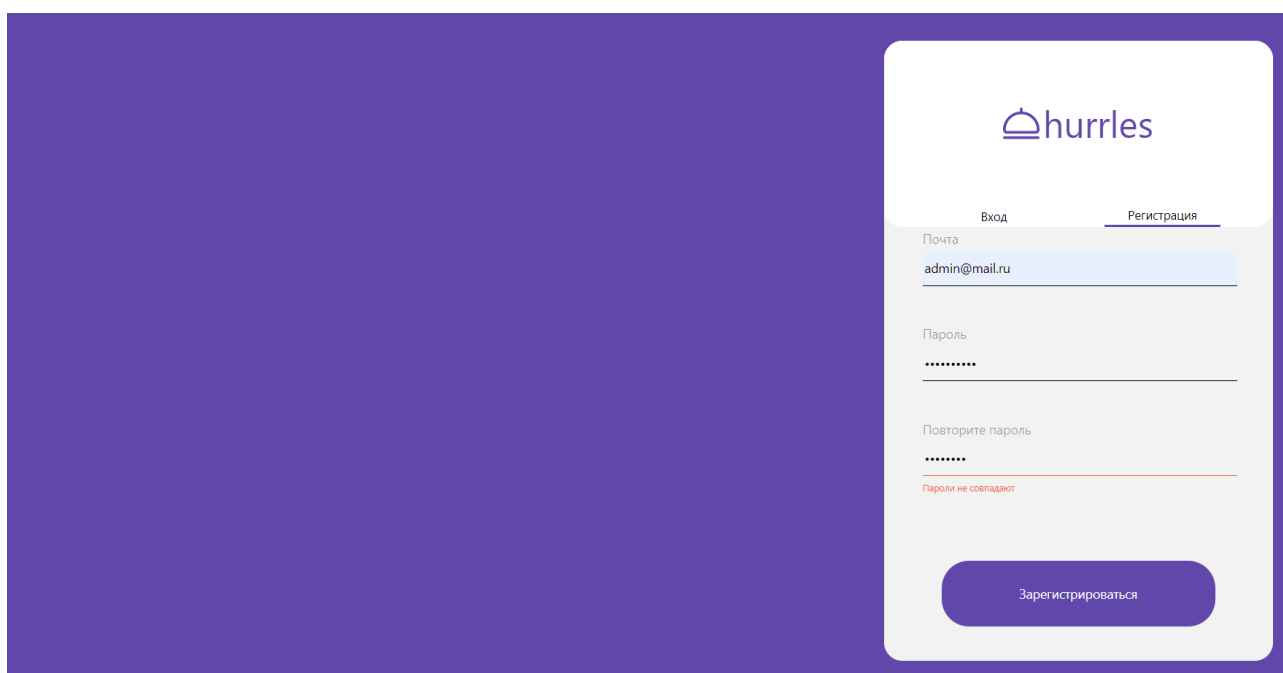
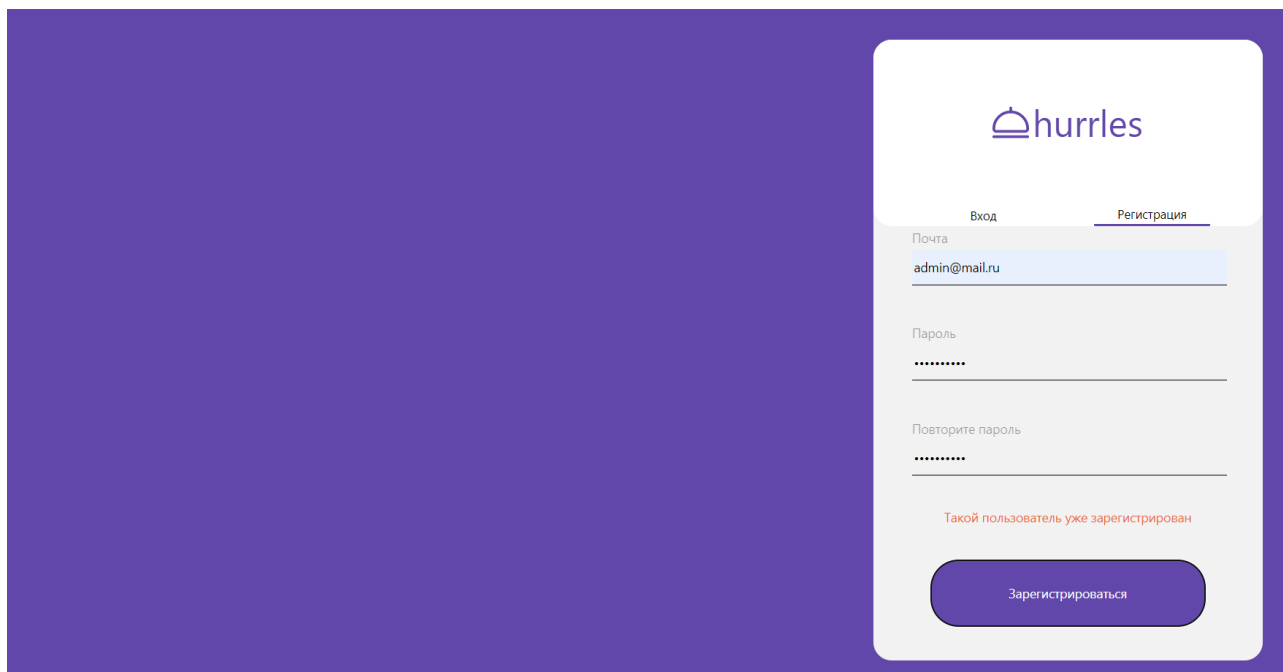


Рис. 10. Страница «Регистрация» с несовпадающим паролем

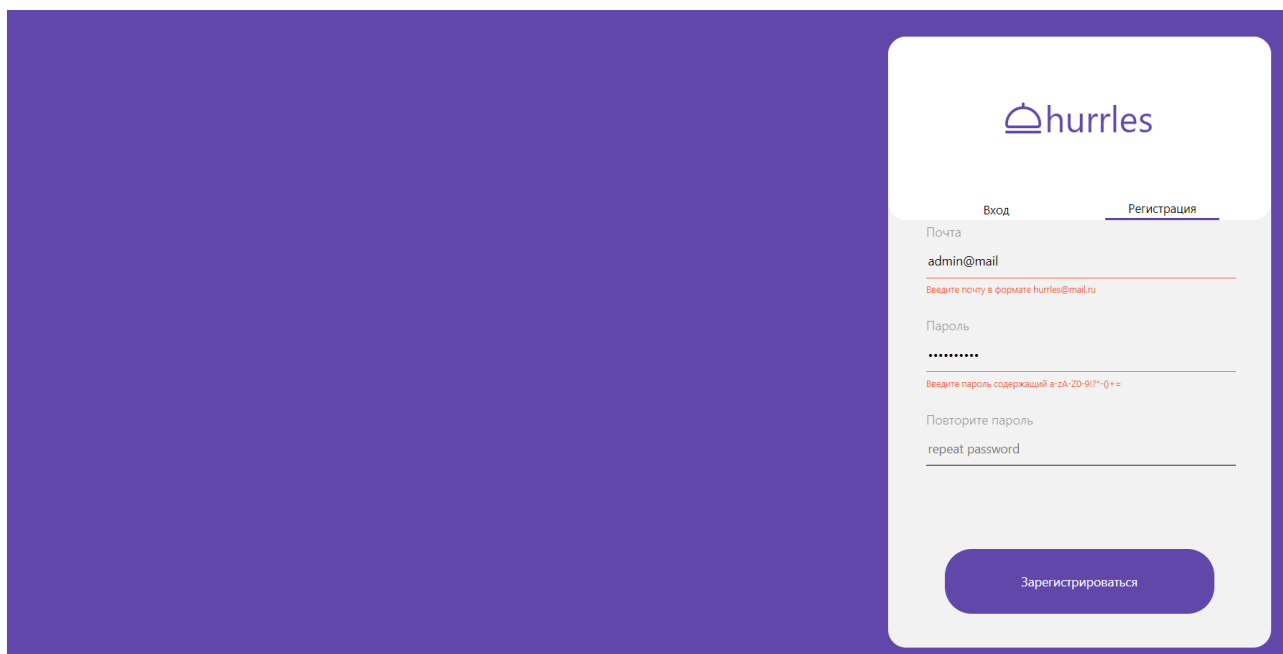
Если все поля валидны, но уже есть такой пользователь, то приложение выдает соответствующую ошибку.



The screenshot shows a registration form on a purple background. The form is titled "hurrlles" and has two tabs: "Вход" (Login) and "Регистрация" (Registration). The "Регистрация" tab is active. The form contains three input fields: "Почта" (Email) with the value "admin@mail.ru", "Пароль" (Password) with masked characters, and "Повторите пароль" (Repeat password) with masked characters. Below the fields, a red message states "Такой пользователь уже зарегистрирован" (Such a user is already registered). At the bottom, there is a blue button labeled "Зарегистрироваться" (Register).

Рис. 11. Страница “Регистрация”

Также существует валидация на почте и пароле.



The screenshot shows the same registration form as in Figure 11, but with validation errors. The "Почта" field has a red error message: "Введите почту в формате hurrlles@mail.ru". The "Пароль" field has a red error message: "Введите пароль содержащий a-zA-Z0-9!~*()-+=". The "Повторите пароль" field has a red error message: "repeat password". The "Зарегистрироваться" button is still present at the bottom.

Рис. 12. Страница “Регистрация” с неправильными почтой и паролем

2.1.2 Тестирование пользовательского функционала

При входе и регистрации мы попадаем на главный экран.

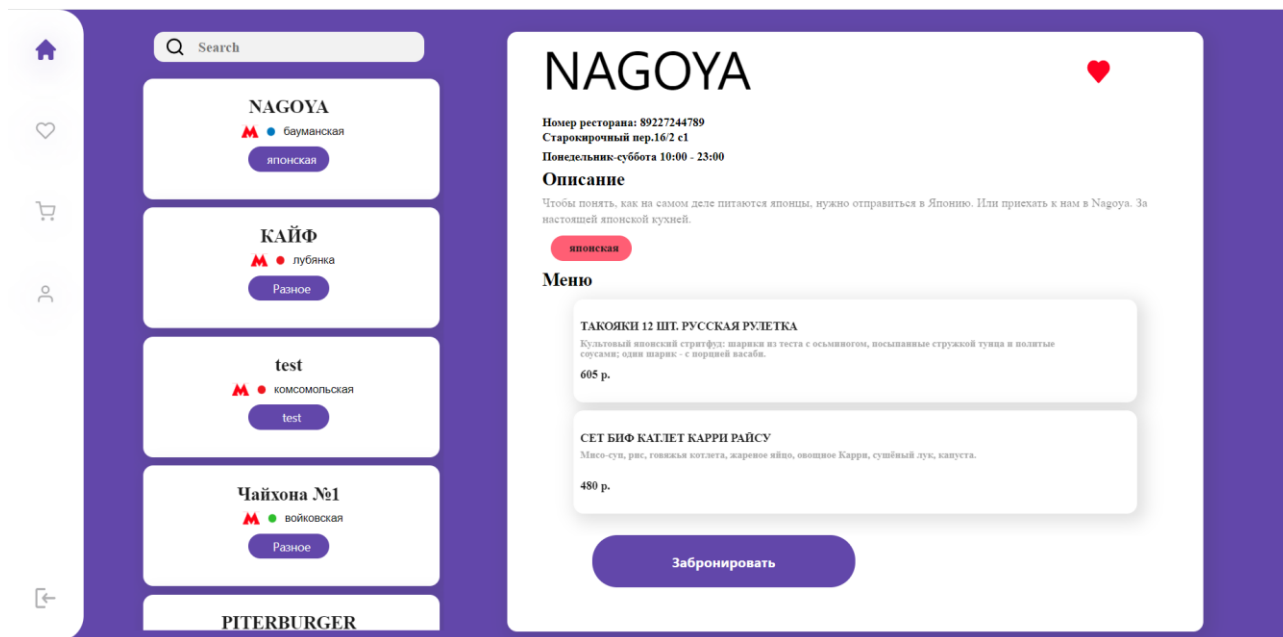


Рис. 13. Страница “Заведения”

Слева находятся кнопки 4 кнопки навигации(тапбар) и кнопка выхода. Кнопка «Заведения» перенесет нас на экран где будут отображаться все ближайшие заведения. Кнопка «Избранное» перенесёт пользователя на экран любимых заведений. Кнопка «Заказы» покажет все открытые ближайшие бронирования. Кнопка «Профиль» покажет информацию профиля, а также позволит редактировать информацию о профиле.

Здесь пользователь может находить заведение общественного питания по названию.

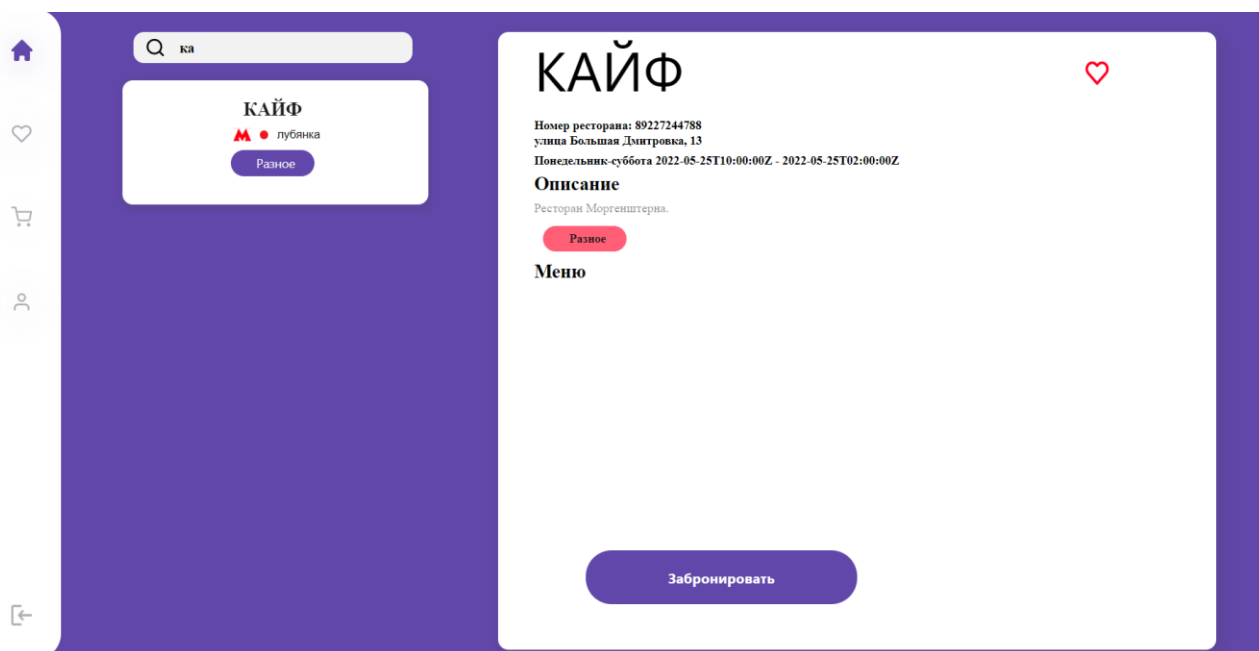


Рис. 14. Страница “Заведения” с поиском

Страница разделяется на 2 части. Слева навигация по заведениям, а справа сама информация выбранного заведения.

При нажатии на кнопку забронировать появляется карта и 2 селектора для выбора этажа и времени бронирования.

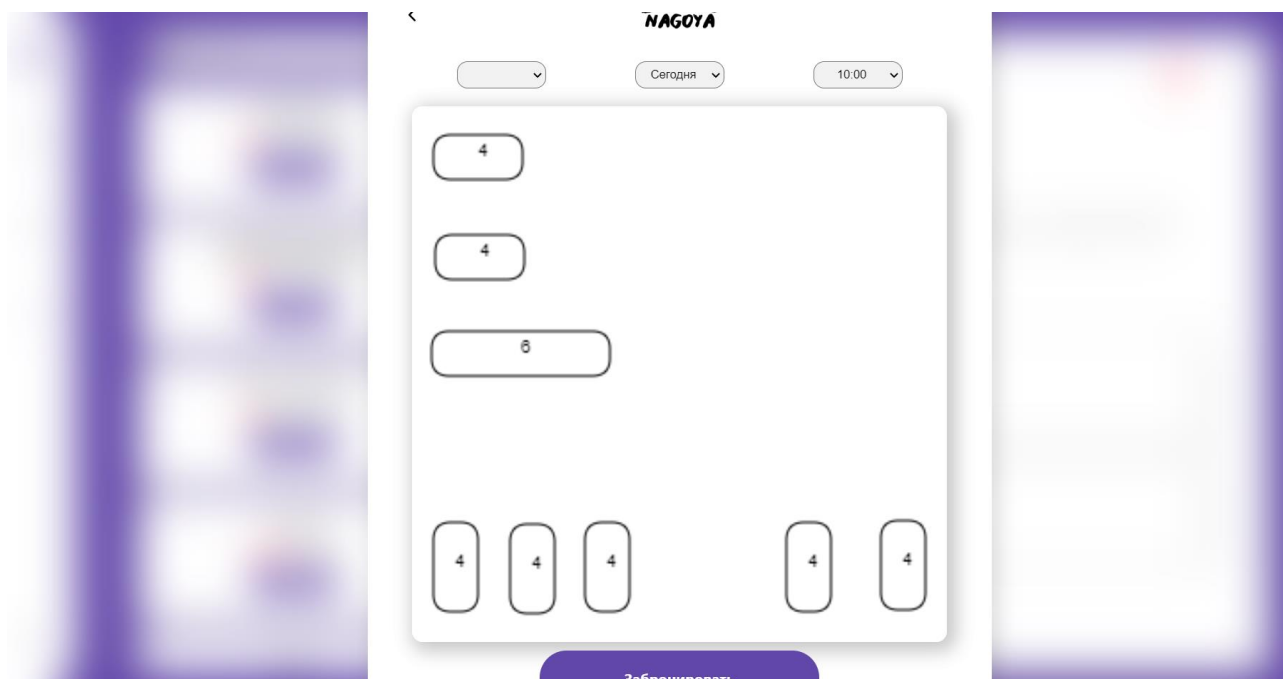


Рис. 15. Страница “Бронирование”

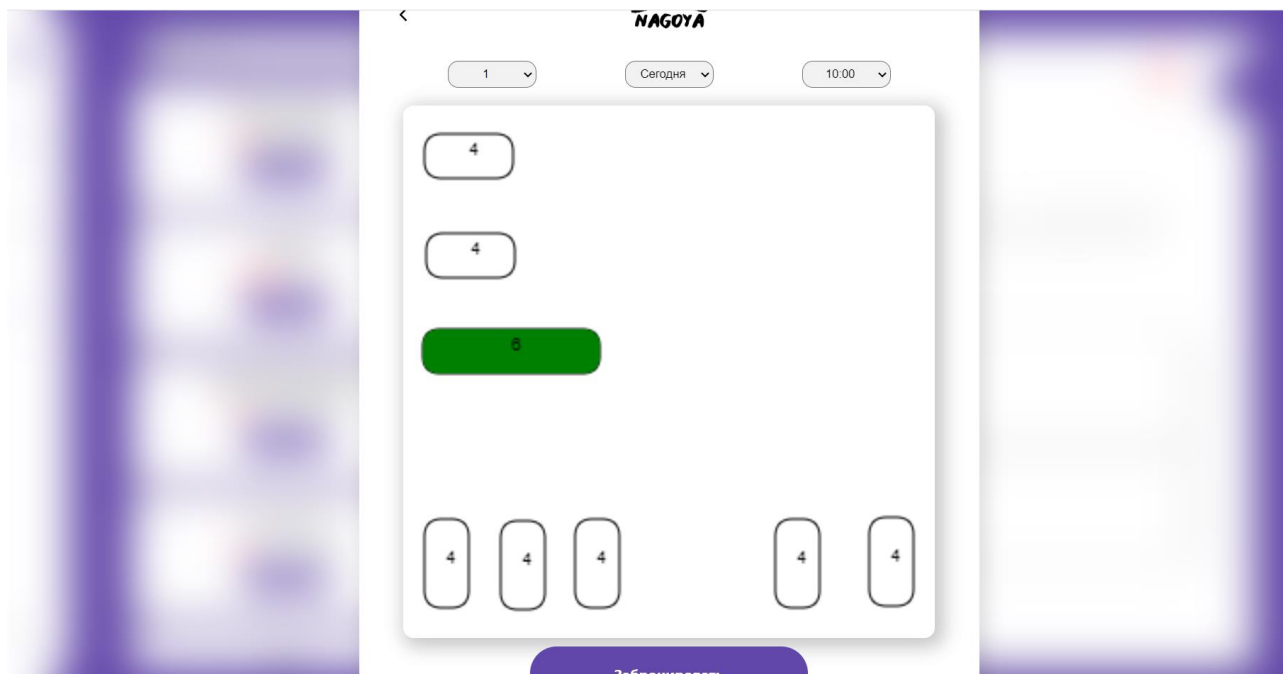


Рис. 16. Страница “Бронирование” с выбранным столиком, местом и временем

После нажатия на кнопку бронирования пользователя переводят на главный экран. Далее он может посмотреть свои заказы по нажатии на тапбаре кнопки «Заказы».

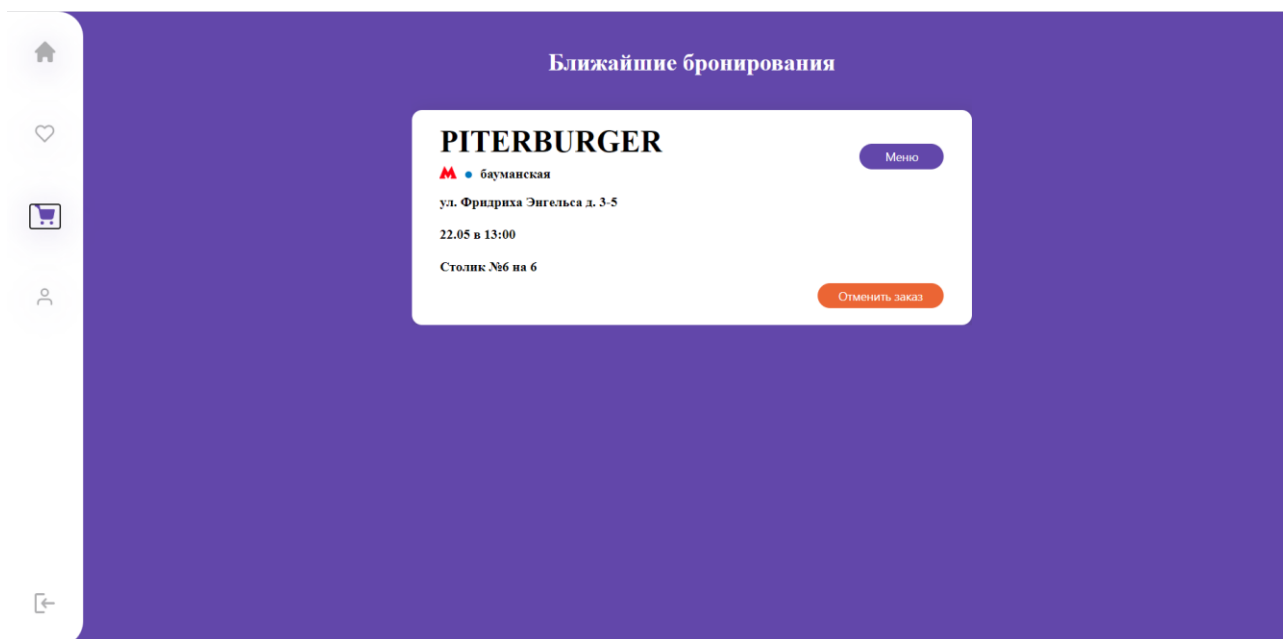


Рис. 17. Страница “Заказы”

По нажатии на кнопку «Меню» у нас отображаются все блюда, которые мы можем добавить в заказ.

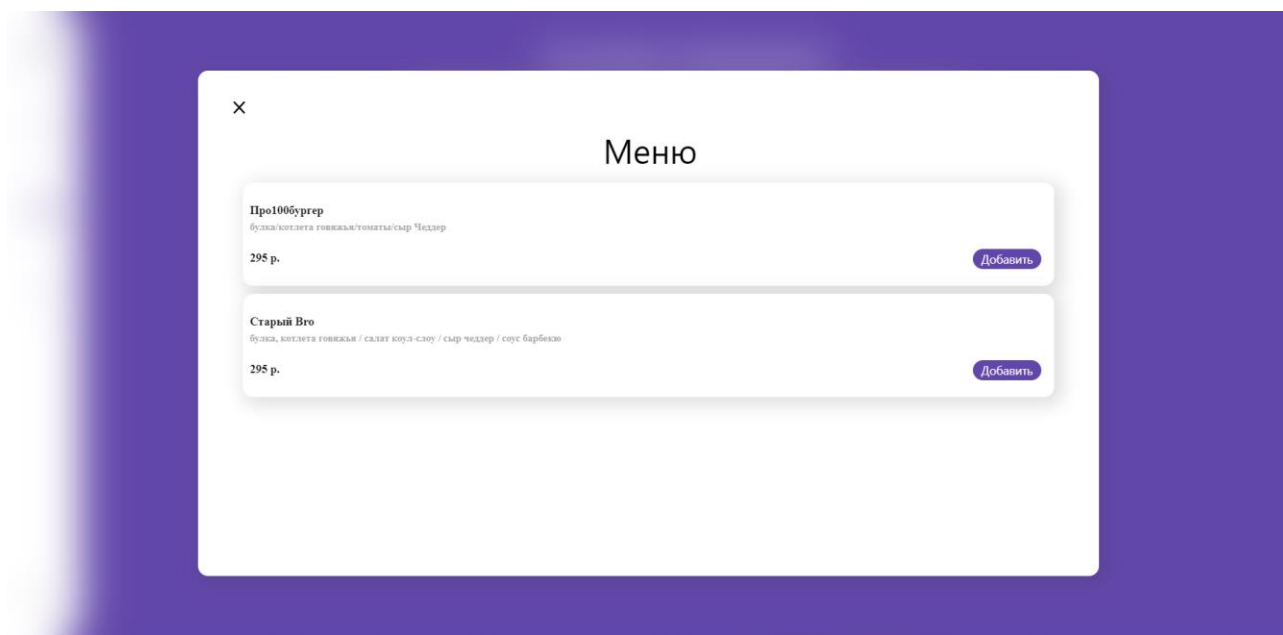


Рис. 18. Страница “Меню”

Как мы видим заказ изменился и пользователю отображаются заказанные блюда и общая стоимость.

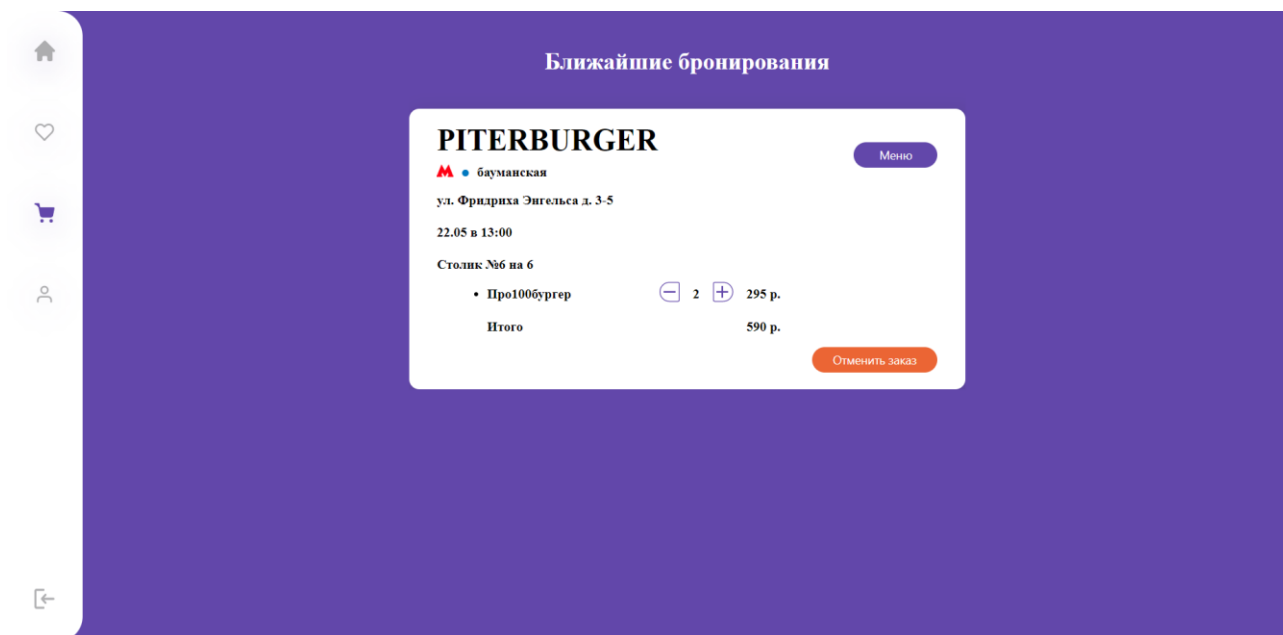


Рис. 19. Страница “Меню” с выбранными блюдами

При переходе на страницу профиль нам отображается вся информация о пользователе, которую можно изменять.

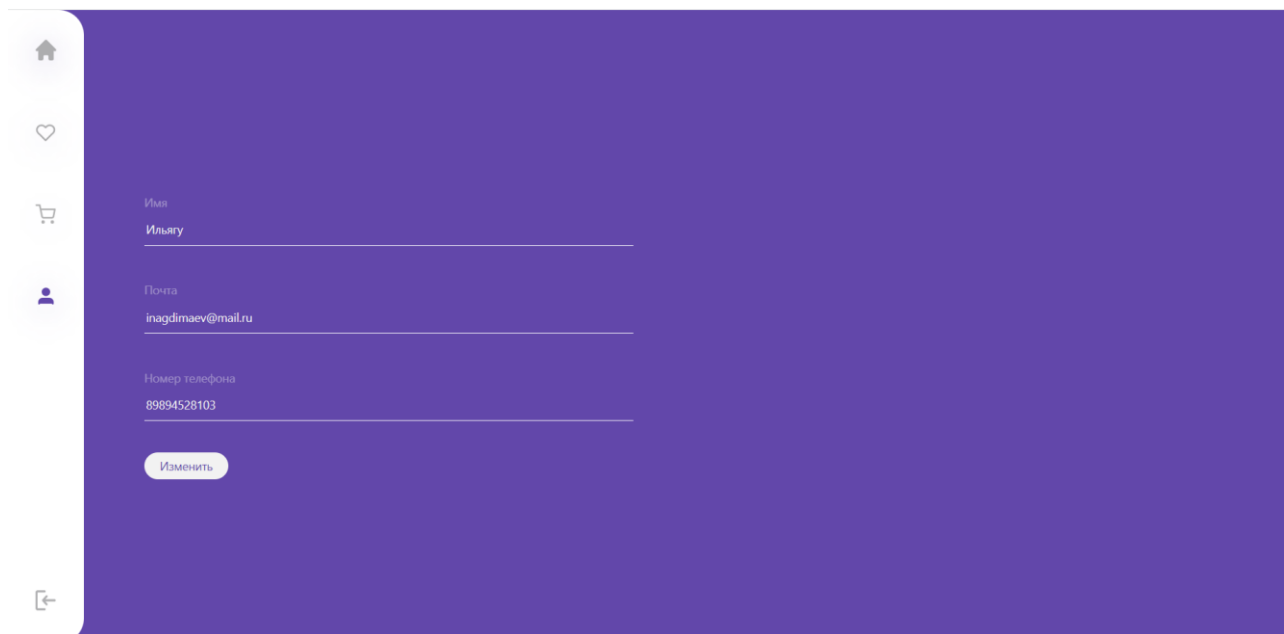


Рис. 20. Страница “Профиль”

При нажатии на кнопку «Избранное» пользователю отобразятся те заведения, которые он пометил по нажатию на кнопку «Лайк».

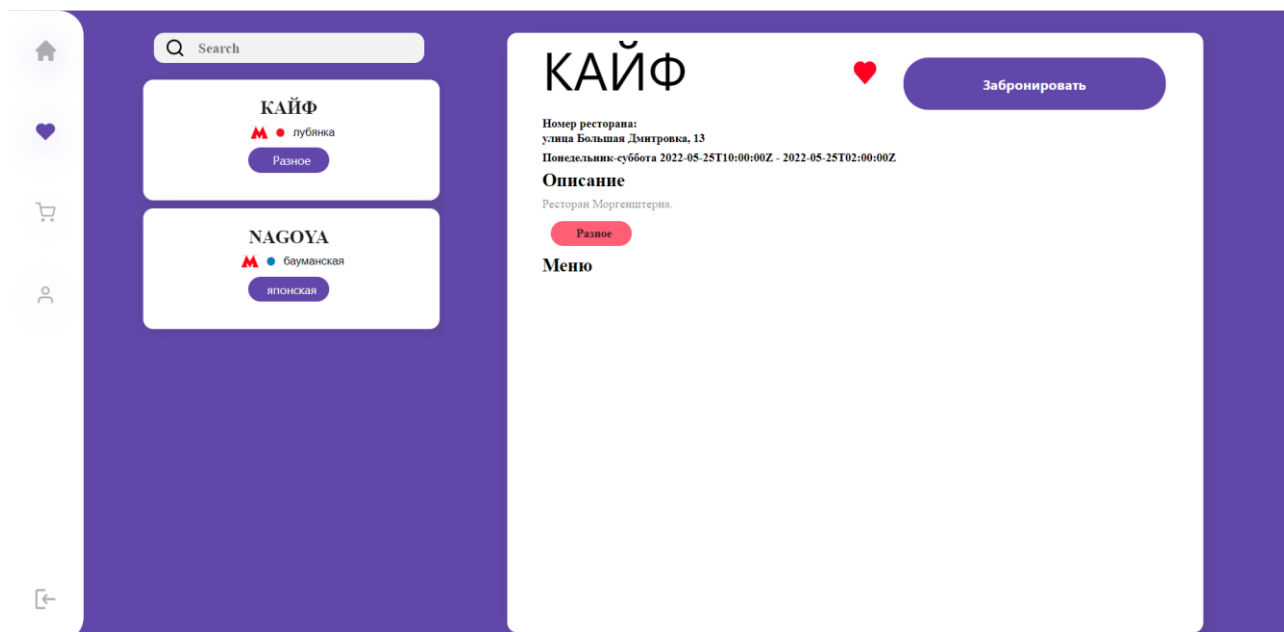


Рис. 21. Страница “Избранное”

При повторном нажатии на кнопку «Лайк» она становится пустой и удаляет заведение из избранного.

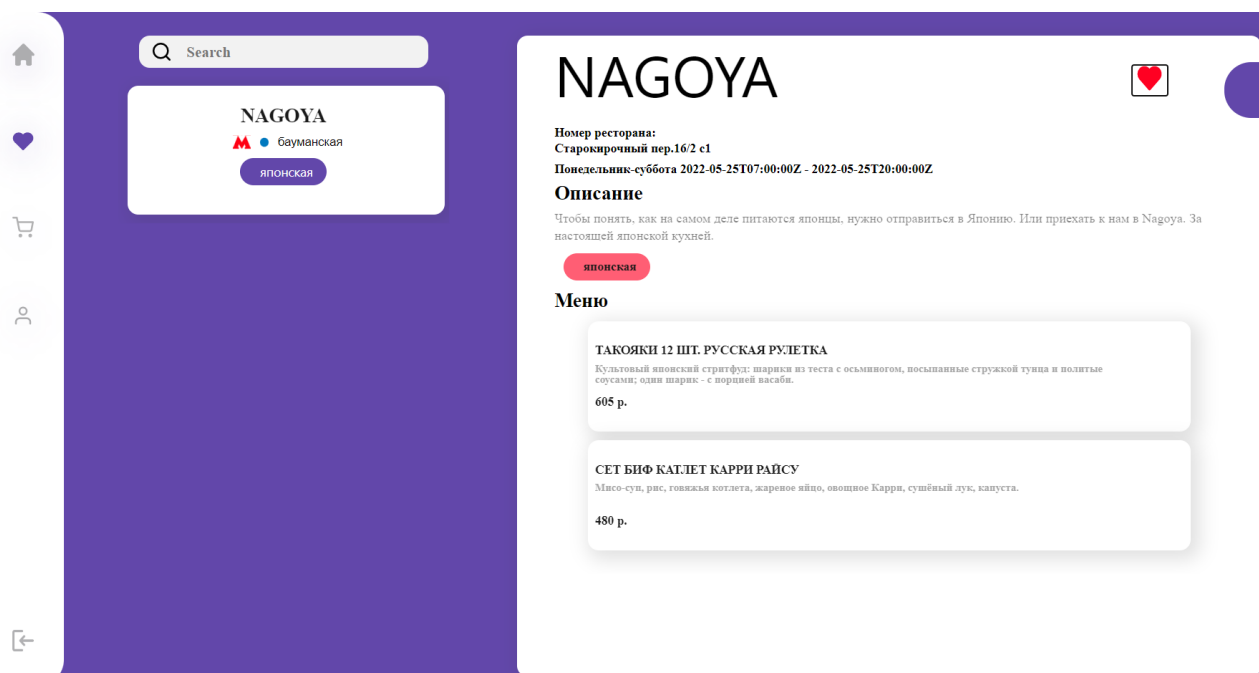


Рис. 22. Страница “Избранное” с удалённым заведением.

2.1.3 Тестирование функционала администратора заведения

При входе в панель заведения появляется страница «Ближайшие заказы»

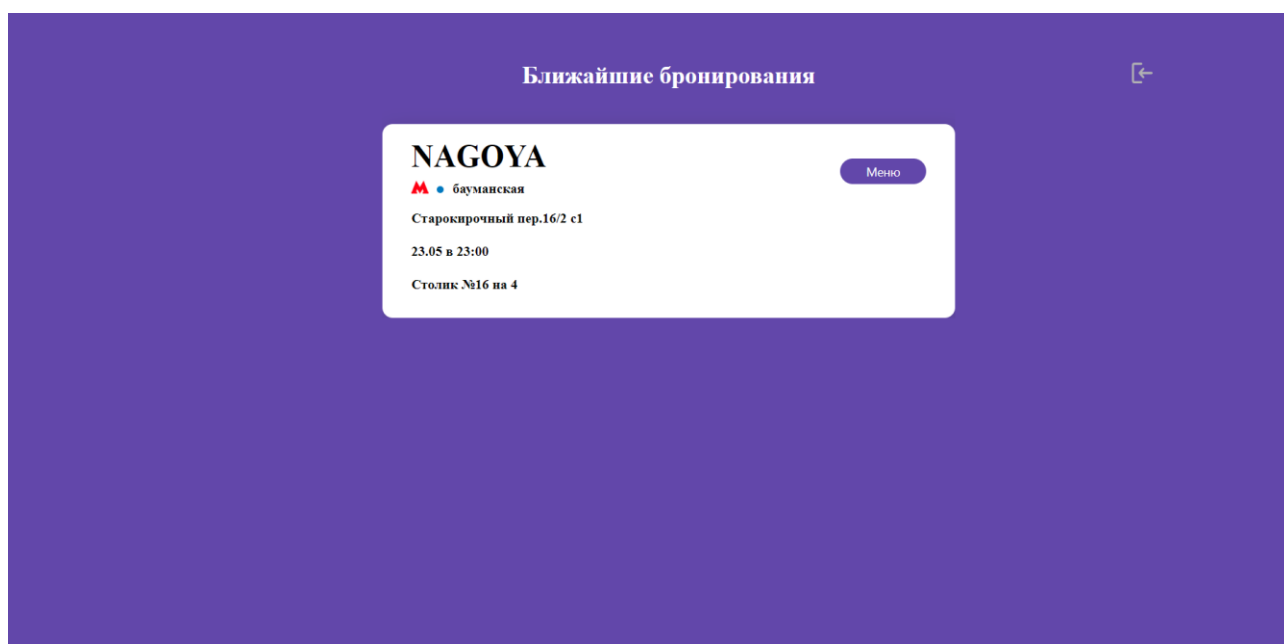


Рис. 23. Страница “Ближайшие заказы”

Здесь отображаются все открытые заказы заведения от пользователей. На рис.25 отображается заказ, который пользователь сделал при тестировании пользовательского интерфейса. Отображается сам заказ, кнопка «Меню» и кнопка выхода.

При нажатии на кнопку «Меню» появляются блюда которые можно добавить в заказ.

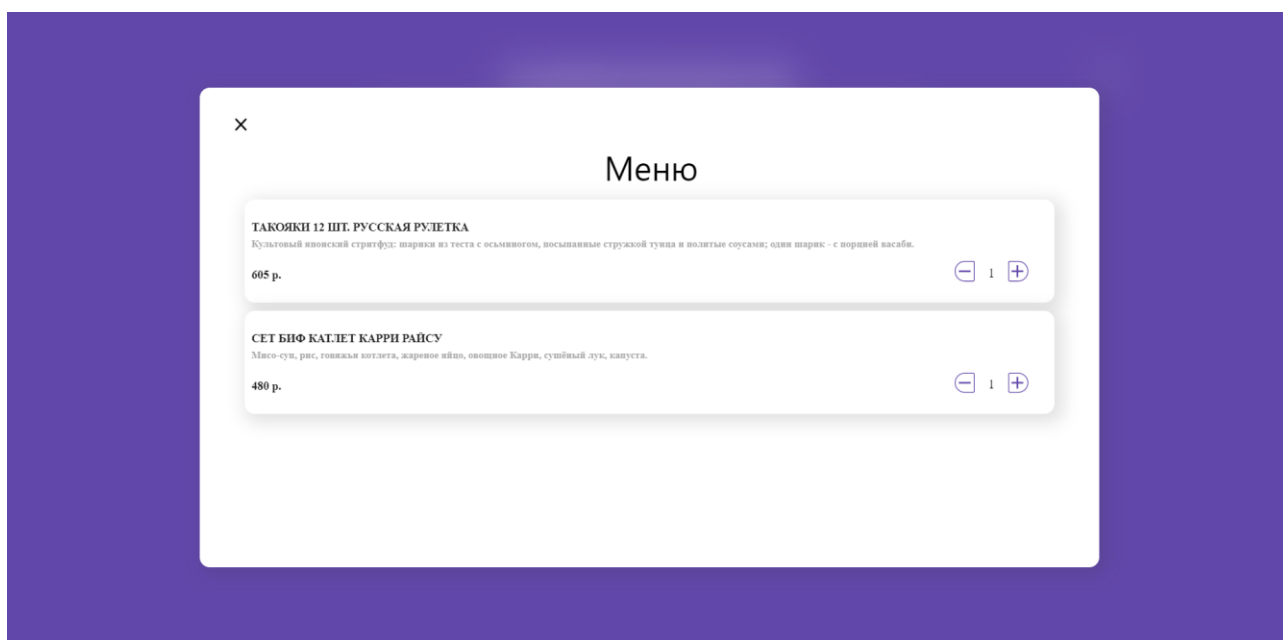


Рис. 24. Страница “Меню сотрудника заведения”

После добавления заказ меняется на соответствующий.



Рис. 25. Страница “Ближайшие заказы” с добавленным меню

2.1.4 Тестирование функционала администратора

При попадании на панель администратора открывается страница «Все заведения»

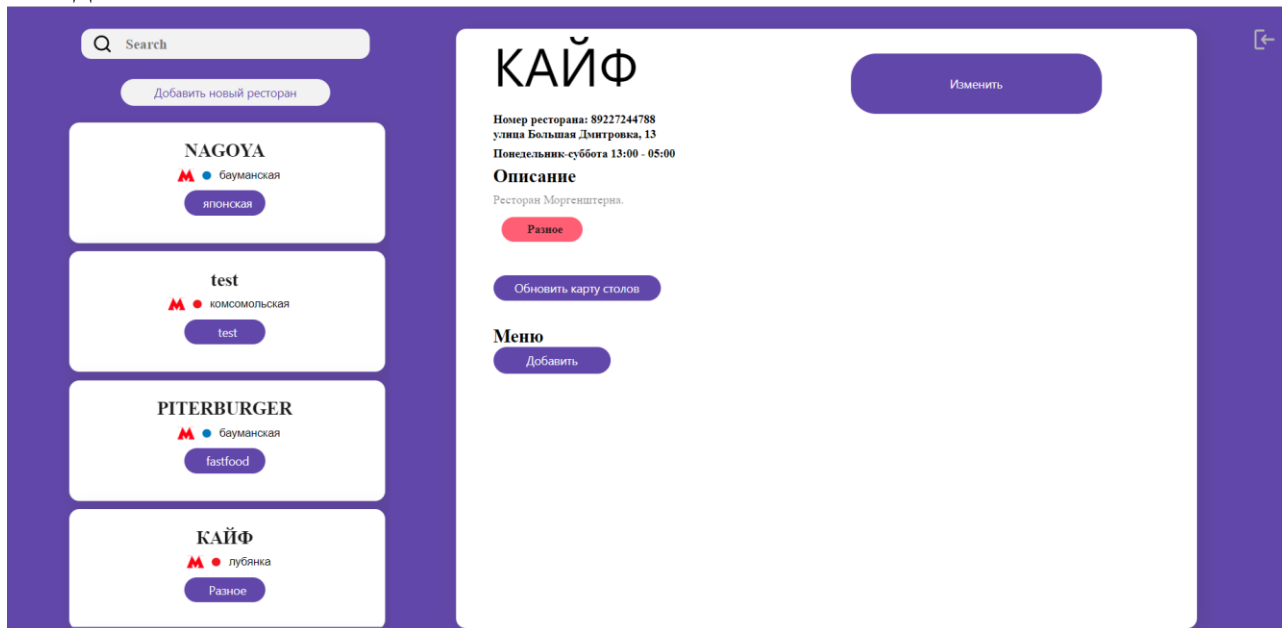


Рис. 26. Страница “Все заведения”

Здесь администратору отображается кнопка «Добавить новое заведение», а на форме заведения отображаются кнопка «Изменить», кнопка «Обновить карту столов», кнопка «Добавить».

При нажатии на кнопку «Добавить новое заведение» отображается форма добавления заведения.

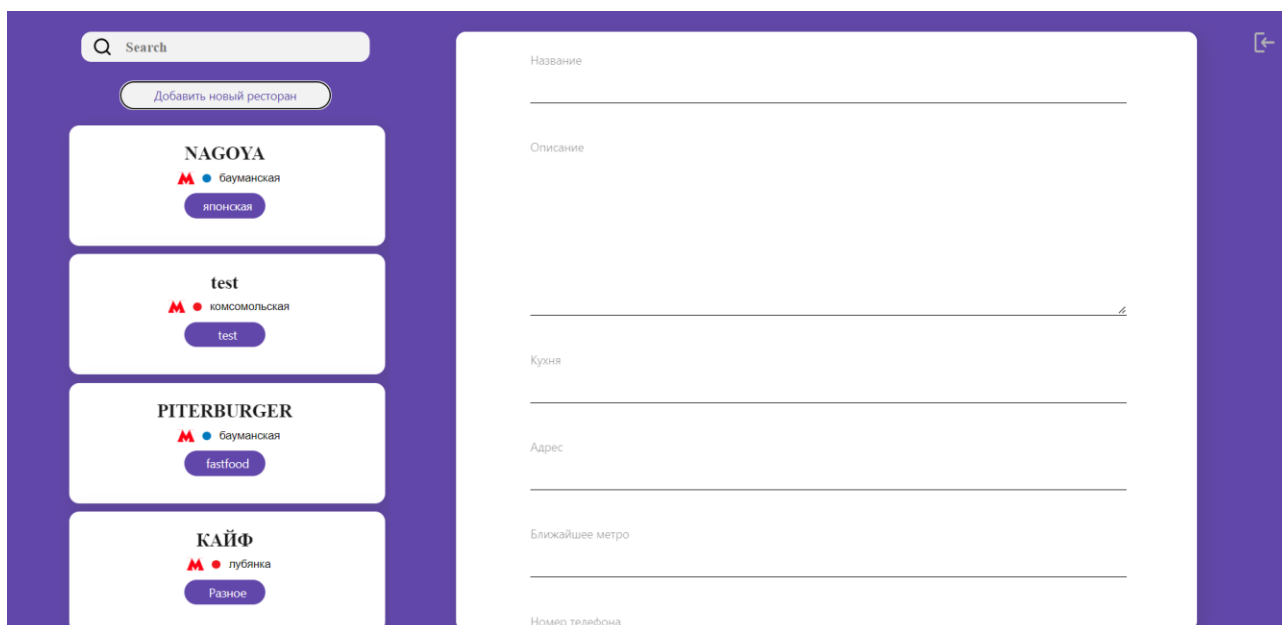


Рис. 27. Форма добавления заведения

При нажатии на кнопку «Изменить» на форме заведения отображается форма для изменения информации о заведении.

The screenshot shows a web application interface with a purple background. On the left, there is a sidebar with a search bar, a 'Добавить новый ресторан' button, and a list of four restaurant cards: NAGOYA (японская), test (комсомольская), PITERBURGER (fastfood), and КАЙФ (Разное). The main area on the right displays the 'Edit Restaurant' form for 'КАЙФ'. The form includes fields for: Название (КАЙФ), Описание (Ресторан Моргенштерна), Кухня (empty), Разное (empty), Адрес (улица Большая Дмитровка, 13), Ближайшее метро (лубянка), and Номер телефона (empty).

Рис. 28. Форма заведения

При нажатии на кнопку «Добавить» на форме заведения отображается форма добавления блюда в меню.

The screenshot shows the same web application interface as Figure 28, but the main area on the right displays the 'Add Dish' form. This form includes fields for: Название (empty), Описание (empty), and Цена (empty). A large purple 'Добавить' button is located at the bottom right of the form area. A back arrow icon is visible in the top right corner of the main area.

Рис. 29. Форма блюда

При нажатии на кнопку «Обновить карту столов» на форме заведения отображается форма редактирования карты.

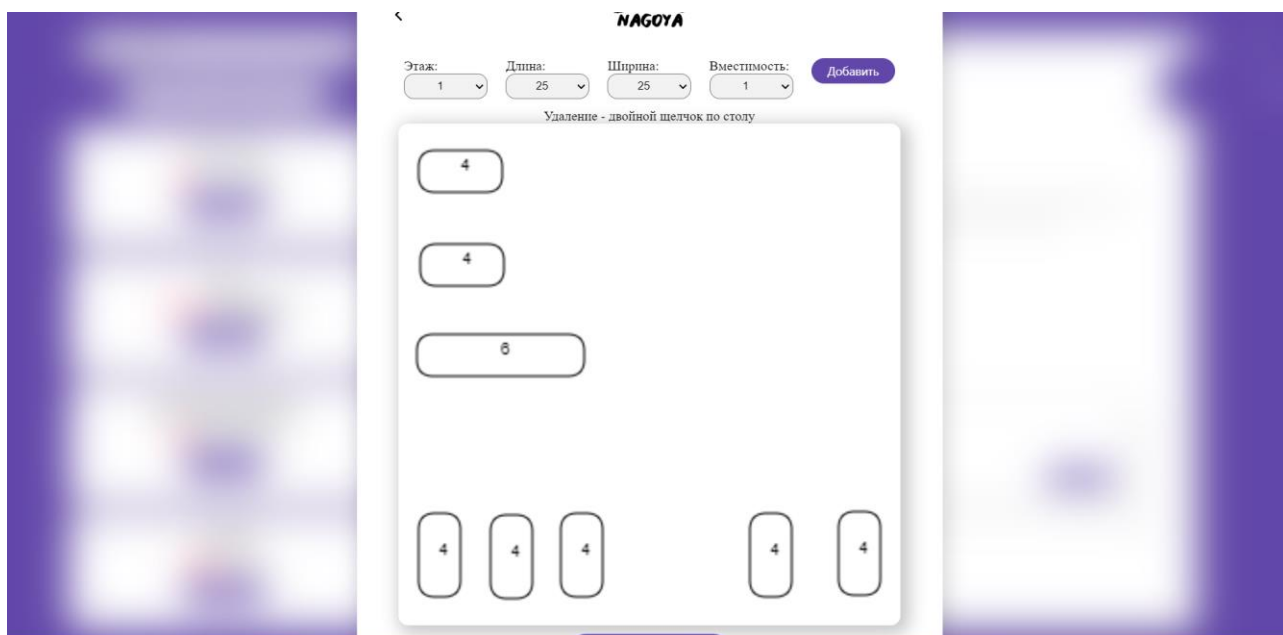


Рис. 30. Форма редактирования карты

При нажатии на кнопку «Добавить» прорисовывается новый стол.

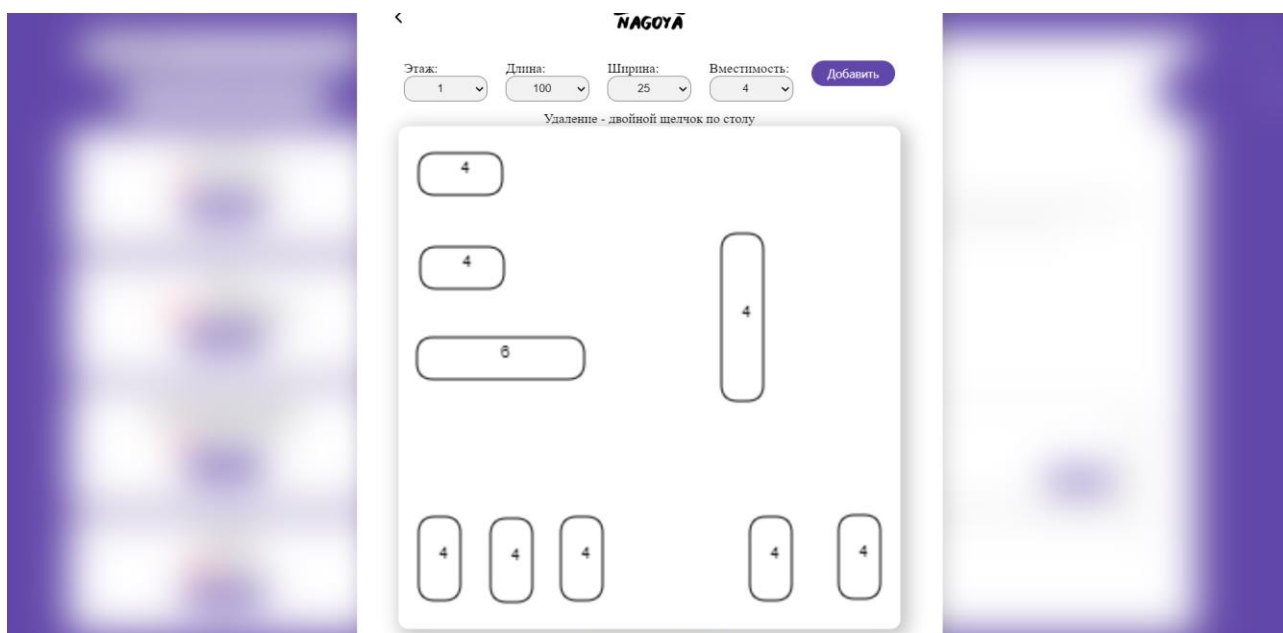


Рис. 31. Форма редактирования карты после добавления стола

ЗАКЛЮЧЕНИЕ

В результате выполнения Выпускной квалификационной работы бакалавра было разработано приложение для резервации заведений общественного питания.

В системе были реализованы следующие возможности для пользователей:

1. Аутентификация клиента.
2. Добавление в БД клиента.
3. Изменения информации о клиенте.
4. Предоставление клиенту доступных заведений.
5. Нахождение заведений по названию.
6. Выбор времени резервирования в заведении.
7. Резервирование столика в заведении.
8. Выбор позиции в меню, предоставляемых заведением.
9. Отмена заказа
10. Сохранение понравившихся заведений.
11. Предоставление доступных заказов.
12. Добавление нового заведения администратором
13. Изменение информации о заведении администратором
14. Добавление позиции в меню в заведении администратором
15. Изменение информации о позиции в меню администратором
16. Добавление столика в заведении администратором
17. Изменение информации о заведении администратором
18. Предоставление заведению ближайших заказов

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. За и против: Зачем заведениям сервисы бронирования столиков
[Электронный ресурс] – URL: <https://habr.com/ru/company/jowi/blog/369115/>
Дата обращения 22.06.2022
2. Стек технологий для разработки веб-приложений: что важно знать бизнесу
[Электронный ресурс] – URL: <https://www.azoft.ru/blog/web-development-stack/>Дата обращения 22.06.2022
3. Что такое SPA, MPA и PWA. Плюсы и минусы популярных подходов к построению сайтов [Электронный ресурс] – URL:
<https://www.azoft.ru/blog/spa-mpa-pwa> Дата обращения 22.06.2022
4. Flux-архитектура [Электронный ресурс] – URL:
<https://ru.wikipedia.org/wiki/Flux-архитектура/> Дата обращения 22.06.2022
5. Чистая архитектура [Электронный ресурс] – URL:
<https://habr.com/ru/post/269589> Дата обращения 22.06.2022
6. JavaScript [Электронный ресурс] – URL:
<https://ru.wikipedia.org/wiki/JavaScript> Дата обращения 22.06.2022
7. React [Электронный ресурс] – URL: <https://ru.wikipedia.org/wiki/React> Дата обращения 22.06.2022
8. React.js: понятное руководство для начинающих [Электронный ресурс] – URL: <https://habr.com/ru/company/ruvds/blog/428077/> Дата обращения 22.06.
9. SCSS — немного практики, часть I [Электронный ресурс] – URL:
<https://habr.com/ru/post/140612/> Дата обращения 22.06.2022
10. Go I [Электронный ресурс] – URL: <https://ru.wikipedia.org/wiki/Go> Дата обращения 22.06.2022
11. PostgreSQL [Электронный ресурс] – URL:
<https://ru.wikipedia.org/wiki/PostgreSQL/> Дата обращения 22.06.2022
12. Tarantool [Электронный ресурс] – URL: <https://ru.wikipedia.org/wiki/Tarantool>
Дата обращения 22.06.2022

13.Docker [Электронный ресурс] – URL: <https://ru.wikipedia.org/wiki/Docker>

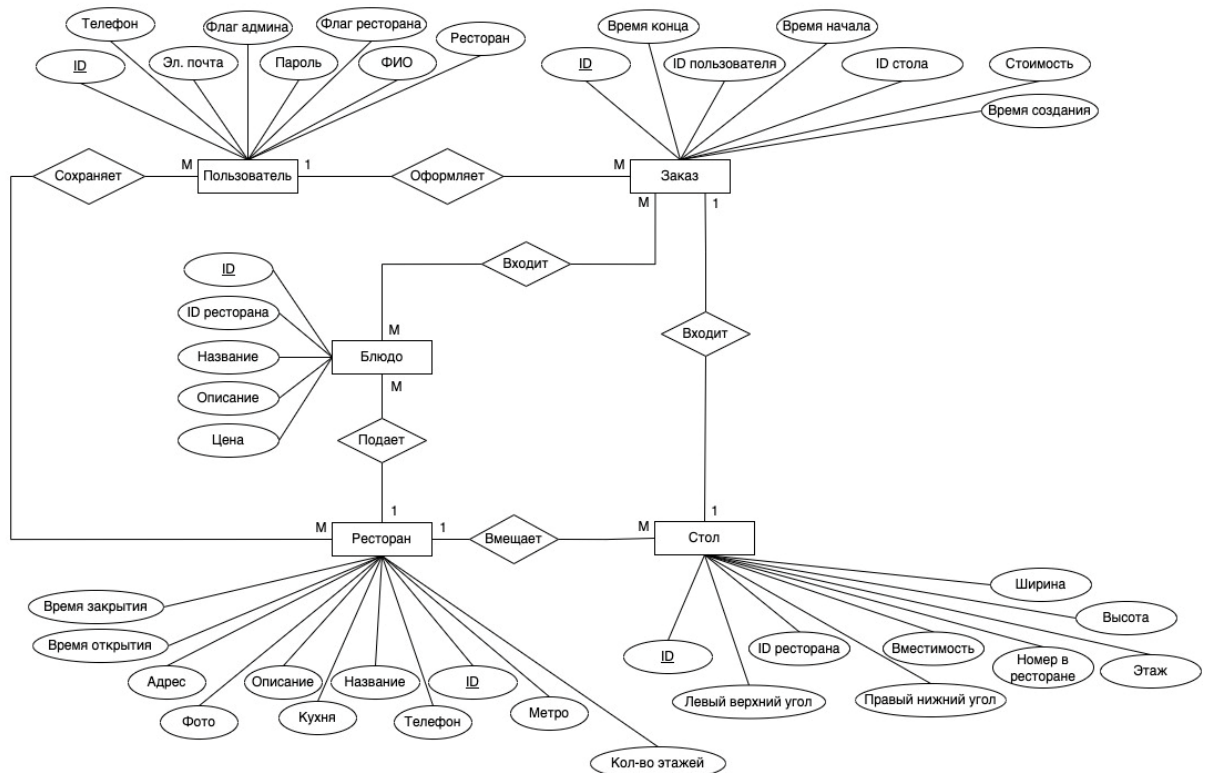
Дата обращения 22.06.2022

14.Canvas (HTML) [Электронный ресурс] – URL:

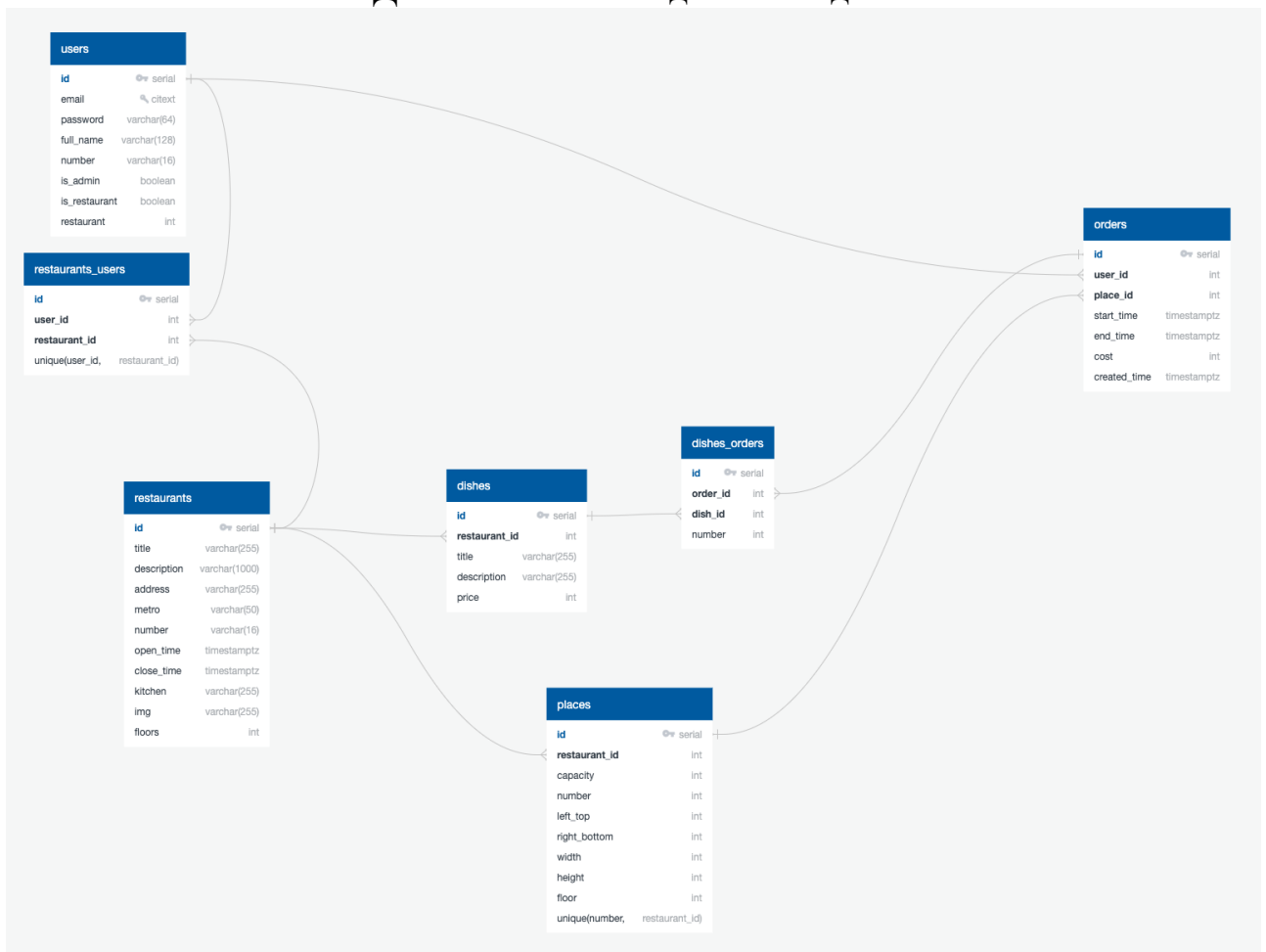
[https://ru.wikipedia.org/wiki/Canvas_\(HTML\)](https://ru.wikipedia.org/wiki/Canvas_(HTML)) ата обращения 22.06.2022

ПРИЛОЖЕНИЕ А
ГРАФИЧЕСКИЕ МАТЕРИАЛЫ

A1 Инфологическая модель в нотации Чена



A2 Даталогическая модель базы данных



ПРИЛОЖЕНИЕ В
ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Кафедра «Системы обработки информации и управления»

Утверждаю
Научный руководитель
_____ Черненко
М.В.
" __ " _____ 2022 г.

Информационная система резервирования мест в ресторанах

Техническое задание
(вид документа)

писчая бумага
(вид носителя)

7
(количество листов)

ИСПОЛНИТЕЛЬ:

_____ Нагдимаев Ильягу Игоревич
" __ " _____ 2022 г.

Москва - 2022

1. Наименование

Информационная система торговой площадки заведений.

2. Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу, подписанное руководителем выпускной работы и утверждённое заведующим кафедрой ИУ5 МГТУ им. Н.Э. Баумана.

3. Исполнитель

Студент четвёртого курса группы ИУ5-85Б Нагдимаев И.И.

4. Назначение и цель работы

Назначение работы – разработка программного обеспечения, позволяющего пользователям резервировать места в заведениях общественного питания, а также предварительно выбирать позиции в меню.

Цель работы – конструирование и разработка информационной системы, позволяющей инструментами веб-разработки разработать программное обеспечение, удовлетворяющее потребностям пользователя в выборе заведения, резервировании столиков и выборе позиций в меню.

5. Содержание работы

5.1 Задачи

5.1.1. Исследовать предметную область, определить функциональные задачи.

5.1.2. Выбрать архитектуру программного обеспечения.

5.1.3. Разработать структуру программного обеспечения.

- 5.1.4. Собрать данные предметной области.
- 5.1.5. Реализовать веб-приложение.
- 5.1.6. Провести тестирование информационно - программного продукта.
- 5.1.7. Провести отладку программного продукта.
- 5.1.8. Оформить техническую документацию.

5.2 Требования к функциональным характеристикам

Разрабатываемая система должна выполнять следующие функции:

- 5.2.1. Авторизация пользователя.
- 5.2.2. Регистрация пользователя.
- 5.2.3. Редактирование профиля.
- 5.2.4. Показ пользователю доступных заведений.
- 5.2.5. Поиск заведения по названию.
- 5.2.6. Выбор времени резервирования в выбранном заведении.
- 5.2.7. Бронирование места в выбранном заведении.
- 5.2.8. Выбор позиций меню, предоставляемых выбранным заведением.
- 5.2.9. Отмена заказа
- 5.2.10. Сохранение понравившихся заведений.
- 5.2.11. Просмотр заказа.
- 5.2.12. Добавление нового заведения через панель админа
- 5.2.13. Редактирование информации о заведении через панель админа
- 5.2.14. Добавление блюд в заведение через панель админа
- 5.2.15. Редактирование информации о блюде через панель админа
- 5.2.16. Добавление мест в заведении через панель админа
- 5.2.17. Редактирование информации о заведении через панель админа
- 5.2.18. Просмотр заведением ближайших заказов

5.3 Требования к архитектуре программного изделия

Требования к архитектуре программного изделия не предъявляются.

5.4 Требования к входным и выходным данным

5.4.1 Требования к входным данным

1. Данные, полученные от пользователя из формы и проверены на корректность:
 - Личные данные (ФИО, электронная почта, номер телефона).
2. Данные, полученные от заведения из форм администратора:
 - Расположение мест заведения.
 - Меню.
 - Описание (адрес, время работы, номер телефона, название станции метро)

5.4.2 Требования к выходным данным

Список заведений общественного питания, свободные места, список позиций в меню, а также номер резервирования.

5.5 Требования к надежности

Требования к надежности не предъявляются.

5.6 Лингвистические требования

Клиентская часть веб-сервиса должна быть русифицирована.

5.7 Требования к составу программных средств

Использование безопасных, лёгких и надежных библиотек и хранилища.

5.8 Требования к составу технических средств

Минимальные системные требования для работы клиентской части:

Устройства поддерживающие браузеры:

1. Google Chrome не ниже 95 версии
2. Mozilla Firefox не ниже 95 версии

6. Этапы работы

График выполнения отдельных этапов работ приведен в соответствии с приказом об организации учебного процесса в 2021/2022 учебном году.

Таблица 1: Этапы разработки

№ п/п	Наименование этапа и содержание работ	Сроки исполнения
1	Исследование предметной области	Апрель 2022 г.
2	Выбор архитектуры и разработка структуры программного обеспечения	Апрель 2022 г.
3	Создание программ	Апрель — Май 2022 г.
4	Тестирование и отладка	Май 2022 г.
5	Оформление документации	Май — Июнь 2022 г.
6	Защита работы	Май 2022 г.

7. Техническая документация

По окончании работы предъявляется следующая техническая документация:

1. Техническое задание;
2. Расчётно-пояснительная записка;
3. Программа и методика испытаний;

8. Порядок приема работы

Приём и контроль программного изделия осуществляется в соответствие с методикой испытаний (см. документ «Программа и методика испытаний»).

9. Дополнительные условия

Данное техническое задание может уточняться в установленном порядке.

ПРИЛОЖЕНИЕ С
ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Кафедра «Системы обработки информации и управления»

Утверждаю
Научный руководитель
_____ Черненко М.В.
" _ " _____ 2022 г.

Информационная система торговой площадки ресторанов

Техническое задание
(вид документа)

писчая бумага
(вид носителя)

9
(количество листов)

ИСПОЛНИТЕЛЬ:

_____ Нагдимаев Ильягу Игоревич
" _ " _____ 2022 г.

Аннотация

В данном документе описываются последовательность и методы проведения испытаний при тестировании программного изделия, состав и структура технических и программных средств, необходимых для проведения испытаний, а также приводятся требования к предъявляемой документации, характеристикам программы применительно к условиям эксплуатации и требования к информационной и программной совместимости.

Объект испытаний

Сервис для агрегирования заведений общественного питания и бронирования столиков.

1. Цель испытаний

Цель испытания – проверка функционирования всех указанных в техническом задании функций программы.

2. Состав предъявляемой документации

На испытания программного продукта предъявляются следующие документы:

- 1) Техническое задание.
- 2) Программа и методика испытаний.

3. Технические требования

4.1 Требования к программной документации

Комплектность программной документации должна удовлетворять разделу данного документа "Состав предъявляемой документации".

4.2 Требования к техническим характеристикам

4.2.1 Требования к составу аппаратного обеспечения

Данная программа должна работать на компьютере следующей конфигурации:

- Устройство ввода
- Устройство вывода

4.2.2 Требование к составу программного обеспечения

Для работы данного модуля необходимо, чтобы на компьютере были установлены следующие программные продукты:

- Microsoft Windows 10 и выше
- Дистрибутив Linux выше 2020 года
- Браузер Google Chrome не ниже 95 версии

- Браузер Mozilla Firefox не ниже 95 версии
- Доступ к сети Интернет 20 Мбит/с

5. Порядок проведения испытаний

Испытания данного программного продукта будут проводиться в следующем порядке:

- 1) Запуск системы. Запуск программного изделия осуществляется в соответствии с инструкциями из пункта 5.3 «Методы испытаний».
- 2) Тестирование базовых функций приложения.

5.1 Требования к составу аппаратного обеспечения

Требования к составу аппаратного обеспечения учитываются, согласно документу, программа и методика испытаний, пункт 4.2.1.

5.2 Требования к составу программного обеспечения

Требования к составу программного обеспечения учитываются, согласно документу, программа и методика испытаний, пункт 4.2.2.

5.3 Методы испытаний

Таблица 1 – Порядок проведения испытаний

№	№ ф-ии из ТЗ	Действие	Результат
1	5.2.1	Открытие веб-приложения в браузере.	Веб-приложение открыто. Отображение страницы «Вход» с интерфейсом для авторизации пользователя. Отображение интерфейса для перехода на страницу «Регистрация».
2	5.2.1	Ввод не валидных почты и пароля в поля «Почта» и «Пароль» и нажатие на кнопку «Войти».	Отображение на странице «Вход» ошибки, сообщающей, что данные неверны.
3	5.2.1	Ввод валидных почты и пароля в поля «Почта» и «Пароль» и нажатие на кнопку «Войти».	Отображение главной страницы приложения «Список доступных заведений». Отображение навигационной панели. Выделение на навигационной панели кнопки главной страницы (первая слева).
4	5.2.3	Нажатие на кнопку аккаунта в навигационной панели (первая справа).	Отображение страницы «Профиль» с информацией о пользователе. Выделение на навигационной панели кнопки профиля (первая справа).
5	5.2.3	На странице «Профиль» ввод имени пользователя в поле имени пользователя, нажатие кнопки «Изменить».	Изменение имени пользователя.
6	5.2.3	На странице «Профиль» нажатие на кнопку выхода из профиля (нижний левый угол).	Отображение страницы «Вход» с интерфейсом для авторизации пользователя. Отображение интерфейса для перехода на страницу «Регистрация».
7	5.2.2	Нажатие на кнопку «Регистрация».	Отображение страницы «Регистрация» с интерфейсом для регистрации пользователя. Отображение интерфейса для перехода на страницу «Вход».
8	5.2.2	Ввод не валидных почты и пароля в поля «Почта», «Пароль» и «Повторите пароль» и нажатие на кнопку «Зарегистрироваться».	Отображение на странице «Регистрация» ошибки, сообщающей, что данные неверны.
9	5.2.2	Ввод валидных почты и пароля в поля «Почта», «Пароль» и «Повторите пароль» и нажатие на кнопку «Зарегистрироваться».	Отображение главной страницы приложения «Список доступных заведений». Отображение навигационной панели. Выделение на навигационной панели кнопки главной страницы (первая слева).
10	5.2.5	На странице «Список доступных заведений» нажатие на поле поиска и ввод названия заведения.	Отображение карточек заведений с совпадающими именами.

11	5.2.4	На странице «Список доступных заведений» нажатие на карточку заведения.	Отображение страницы «Описание заведения» с информацией о заведении и его меню. Отображение кнопки «Забронировать».
12	5.2.7	На странице «Описание заведения» нажатие на кнопку забронировать.	Отображение страницы «Бронирование» с интерфейсом для выбора места и времени бронирования.
13	5.2.6	На странице «Бронирование» выбор времени резервирования.	Отображение свободных мест в заведении на выбранное время.
14	5.2.7	На странице «Бронирование» выбор места резервирования	Выделение выбранного места.
15	5.2.7	На странице «Бронирование» нажатие на кнопку забронировать.	Оформление бронирования. Возврат на страницу «Описание заведения».
16	5.2.10	На странице «Описание заведения» нажатие на кнопку «Сердечко» (верхний правый угол).	Заполнение кнопки «Сердечка» красным цветом. Добавление заведения в избранные.
17	5.2.10	Нажатие на кнопку избранное в навигационной панели (вторая слева).	Отображение страницы «Избранное» с заведениями, добавленными пользователем в избранное. Выделение на навигационной панели кнопки избранное (вторая слева).
18	5.2.11	Нажатие на кнопку заказов в навигационной панели (вторая справа).	Отображение страницы «Заказы» с информацией о текущем заказе пользователя. Выделение на навигационной панели кнопки заказов (вторая справа).
19	5.2.8	На странице «Заказы» нажатие на кнопку «Меню» на выбранном заказе.	Отображение окна «Меню» с информацией о меню заведения.
20	5.2.8	В окне «Меню» нажатие на кнопку «Добавить» у блюда.	Добавление блюда к заказу. Отображение количества добавленных блюд в заказ.
21	5.2.8	На странице «Заказы» нажатие на кнопку «Добавить» (плюс) у блюда.	Увеличение количества добавленных блюд в заказе.
22	5.2.8	На странице «Заказы» нажатие на кнопку «Удалить» (минус) у блюда.	Уменьшение количества добавленных блюд в заказе.
23	5.2.9	На странице «Заказы» нажатие на кнопку «Отменить заказ»	Отмена заказа.
24	5.2.12	Вход за админа.	Отображение панели админа.

25	5.2.12	Нажатие на кнопку «Добавить новое заведение».	Открытие формы для добавления нового заведения.
26	5.2.12	Заполнение формы добавления нового заведения и нажатие на кнопку «Добавить».	Создание нового заведения.
27	5.2.12	Нажатие на карточку заведения.	Отображение информации о выбранном заведении.
28	5.2.13	Нажатие на кнопку «Изменить».	Открытие формы изменения информации о заведении.
29	5.2.13	Изменение информации о заведении и нажатие на кнопку «Изменить».	Изменение информации о заведении. Закрытие информации о заведении.
30	5.2.14	На карточке с информацией о заведении нажатие на кнопку «Добавить» в разделе «Меню».	Открытие формы добавления блюда.
31	5.2.14	Заполнение формы добавления блюда и нажатие на кнопку «Добавить».	Добавления позиции в меню заведения. Закрытие информации о заведении.
32	5.2.15	На карточке с информацией о заведении нажатие на кнопку «Изменить» у блюда.	Открытие формы редактирования информации о блюде.
33	5.2.15	Изменение информации о блюде и нажатие на кнопку «Изменить».	Изменение информации о блюде. Закрытие информации о заведении.
34	5.2.16	На карточке с информацией о заведении нажатие на кнопку «Обновить карту столов».	Открытие формы для добавления и редактирования карты столов в заведении.
35	5.2.16	Нажатие на кнопку «Добавить», установка столика в нужное место и нажатие на кнопку «Сохранить».	Добавление информации о местах в заведении. Закрытие формы обновления карты столов.
36	5.2.17	На форме обновления карты столов изменение расположения, количества, данных столов и нажатие на кнопку «Сохранить».	Изменение информации о местах в заведении. Закрытие формы обновления карты столов.
37	5.2.18	Вход за админа заведения.	Отображение всех будущих заказов заведений.

6. Результат испытаний

Основой испытаний является проверка работы основных функций приложения: отображение пользовательского интерфейса для распознавания

графических образов на изображениях с помощью различных моделей компьютерного зрения. Испытание считается пройденным успешно, если в его процессе пользователь смог успешно зарегистрировать и забронировать столик.